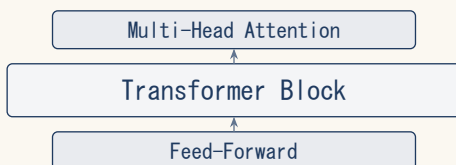


Make LLM basic

トークナイザーから変圧器まで
最初から自分で作る大型言語モデル

dirmich
Highmaru Press



Contents

用語集	ii
ヘッダー	v
1 LLMとは何ですか	1
1.1 言語モデルとは	1
1.2 直観：言語モデルをどう思うか	1
1.3 LLMの発展の歴史	1
1.3.1 統計的言語モデル（1990年代 - 2000年代）	1
1.3.2 ニューラルネットワーク言語モデル（2003 - 2017）	2
1.3.3 トランスフォーマーの登場（2017）	2
1.3.4 GPT シリーズとスケールリング法則（2018 - 現在）	3
1.3.5 アライメントの登場（2022 - 現在）	3
1.4 LLMの主な応用	3
1.4.1 自然言語処理の伝統的なタスク	3
1.4.2 生成タスク	3
1.4.3 専門分野	3
1.4.4 マルチモーダル	4
1.5 なぜ自分で作るのか	4
1.6 この本が作るモデル	4
1.7 学習ロードマップ	4
1.8 要約	4
1.9 練習問題	5
2 開発環境設定	6
2.1 必要なツール	6
2.2 PyTorch インストール	6
2.3 ストレージ構造	6
2.4 最初の例：テンソルを扱う	7
2.5 再現性の確保	7
2.6 ユニットテストの実行	8
2.7 要約	8
3 トークナイザーの作成	9
3.1 なぜトークナイザーが必要なのか	9
3.1.1 3つのアプローチの比較	9
3.2 特殊トークン	9

3.3	ベースクラス	10
3.4	BPE トークナイザー	11
3.4.1	アルゴリズム直観	11
3.4.2	なぜバイト単位なのか?	12
3.4.3	実装: 学習	12
3.4.4	エンコーディング	13
3.4.5	前処理: スペース処理	13
3.4.6	デコード: バイトの復元	14
3.5	Unigram トークナイザー	14
3.5.1	直観	14
3.5.2	アルゴリズム	14
3.5.3	Viterbi デコード	15
3.6	BPE vs Unigram 比較	15
3.7	トークナイザーテスト	15
3.8	練習: 実際の馬でトークナイザーを学ぶ	16
3.9	要約	17
3.10	練習問題	17
4	埋め込みとポジショナルエンコーディング	18
4.1	なぜ埋め込みが必要なのか	18
4.2	埋め込みの数学的定義	18
4.3	Token Embedding 実装	18
4.3.1	padding_idxの役割	19
4.4	ポジショナルエンコーディング	19
4.4.1	直観: なぜ順序が重要なのか?	19
4.4.2	Sinusoidal 位置エンコーディング (Vaswani et al. 2017)	19
4.4.3	学習可能な位置埋め込み (GPT-2スタイル)	20
4.5	埋め込み合成	21
4.5.1	なぜスケールリングが必要なのか?	21
4.6	Layer Normalization はじめに	21
4.7	テスト	22
4.8	練習: 埋め込み可視化	22
4.9	要約	22
4.10	練習問題	22
5	アテンション機構	24
5.1	直観: アテンションとは何ですか	24
5.1.1	直感的なたとえ話: 図書館検索	24
5.2	Q, K, Vどこから来るの?	24
5.3	スケールリング: なぜ - に分けるか?	25
5.4	Scaled Dot-Product Attention 実装	25
5.5	因果マスク (Causal Mask)	25
5.5.1	なぜ - 認可?	26
5.6	Multi-Head Attention	26
5.6.1	アイデア	26

5.6.2	なぜ頭を分けるのか?	26
5.6.3	実装	26
5.7	因果性テスト	27
5.8	Self-Attention vs Cross-Attention	28
5.8.1	クロスアテンションはどう違いますか?	28
5.9	アテンション計算量分析	28
5.10	練習: アテンション重みの可視化	28
5.11	要約	29
5.12	練習問題	29
6	トランスフォーマーブロックの組み立て	30
6.1	変圧器ブロックの構造	30
6.2	Feed-Forward Network (FFN)	31
6.2.1	直観: なぜ拡張-縮小?	31
6.3	GELU vs ReLU	31
6.3.1	なぜ GELUもっと良い?	31
6.4	残差接続(Residual Connection)	32
6.4.1	直観: なぜ残差が役に立つのか?	32
6.5	Pre-LN vs Post-LN	32
6.5.1	Post-LN (Vaswani オリジナル)	32
6.5.2	Pre-LN (GPT-2スタイル)	32
6.5.3	なぜ Pre-LNこれはより安定していますか?	33
6.6	残差スケールリング	33
6.6.1	なぜ - 認可?	33
6.7	ブロックテスト	34
6.8	変圧器ブロックの計算量	34
6.9	要約	34
6.10	練習問題	34
7	GPT モデルの完成と学習ループ	36
7.1	GPT モデル構造	36
7.2	ウェイトタイ (Weight Tying)	36
7.3	Forward Pass	37
7.4	データセット: スライドウィンドウ	37
7.5	損失関数: Cross-Entropy	38
7.5.1	Label Smoothing (選択)	39
7.6	オプティマイザー: AdamW	39
7.7	学習率スケジューラ	39
7.7.1	なぜ warmupこれが必要ですか?	40
7.8	グラデーションクリッピング	40
7.9	トレーナー	40
7.10	Perplexity	41
7.11	チェックポイントを保存/ロード	41
7.12	要約	42
7.13	練習問題	42

8	テキスト生成 - サンプルング戦略とデコード	43
8.1	自己回帰生成の原理	43
8.2	Greedy サンプルング	43
8.3	Temperature サンプルング	44
8.4	Top-K サンプルング	44
8.4.1	Top-Kの直観	45
8.5	Top-P (Nucleus) サンプルング	45
8.5.1	Top-K vs Top-P	46
8.6	サンプルング戦略の比較	46
8.7	生成関数の実装	46
8.8	コンテキスト長制限	47
8.9	サンプラーテスト	47
8.10	練習: 各種サンプルング比較	47
8.11	高度な復号技術(紹介)	48
8.11.1	Beam Search	48
8.11.2	Speculative Decoding	48
8.11.3	Contrastive Search	48
8.12	要約	48
8.13	練習問題	48
9	クリーンアップと次のステップ	49
9.1	1巻まとめ	49
9.1.1	実装したもの	49
9.1.2	テスト	49
9.1.3	読者が持っているもの	49
9.2	1巻の限界	50
9.3	2巻で扱うこと	50
9.3.1	分散学習(2 - 第4章)	50
9.3.2	最新アーキテクチャ(5 - 第6章)	50
9.3.3	量子化と推論の最適化(7章)	50
9.3.4	ソートとファインチューニング(8枚)	51
9.3.5	データパイプライン(9章)	51
9.4	推奨学習パス	51
9.5	最後に	51
A	数学基礎補足	52
A.1	ベクトルと行列	52
A.2	Softmax	52
A.3	Cross-Entropy	52
A.4	連鎖法則と Backpropagation	52
A.5	行列微分	53
A.6	正規分布	53
A.7	情報理論の基礎	53
A.8	最適化の基礎	53
B	コードの完全なリスト	54

B.1	プロジェクト構造	54
B.2	テストの実行	54
B.3	クイックスタートの例	55
B.4	モジュール別 API 要約	56
B.5	設定データクラス	56
B.6	ライセンス	56
C	よくある質問と罣	57
C.1	環境関連	57
C.1.1	Q: PyTorch インストール後 CUDAが認識されません。	57
C.1.2	Q: 学習が遅すぎる	57
C.2	トークナイザー関連	57
C.2.1	Q: BPEで学んだトークナイザー “こんにちは” 奇妙なトークン化	57
C.2.2	Q: encode/decode ラウンドトリップが失敗しました。	57
C.2.3	Q: 特殊トークン IDが変わるとどうなりますか？	58
C.3	モデル関連	58
C.3.1	Q: アテンション出力 NaNこれが出ます。	58
C.3.2	Q: モデルは同じ単語を繰り返します。	58
C.3.3	Q: コンテキスト長を超えるとエラーが発生します	58
C.4	学習関連	58
C.4.1	Q: 損失は減りません。	58
C.4.2	Q: 学習の初期に損失が発散します。	59
C.4.3	Q: チェックポイントをロードすると他の結果が表示されます。	59
C.5	分散学習関連 (2巻プレビュー)	59
C.5.1	Q: DDPで学習すると速度がむしろ遅くなります。	59
C.5.2	Q: FSDPから OOMこれは私です	59
C.6	デバッグのヒント	59
C.6.1	段階的に検証する	59
C.6.2	shapeを常に出力する	60
C.6.3	小さな入力で始める	60
C.7	パフォーマンス最適化のヒント	60
C.7.1	混合精度の使用	60
C.7.2	torch.compile	60
C.7.3	プロファイリング	60
D	練習問題の答え	61
D.1	第1章答え	61
D.2	第3章 答え	61
D.3	第4章 答え	62
D.4	第5章 答え	62
D.5	第6章答え	62
D.6	第7章答え	63
D.7	第8章 答え	63
E	高度なトピックのプレビュー	64
E.1	分散学習	64

E. 2	最新のアーキテクチャ	64
E. 3	量子化と推論の最適化	64
E. 4	ソートとファインチューニング	65
E. 5	データパイプライン	65
E. 6	2巻学習準備	65
E. 7	推奨学習パス	65

用語集

この用語集は、本書で使用する主な用語をアルファベット順にまとめたものです。各項目は、短い定義で対

Activation (アクティブ値)

ニューラルネットワークの各層を通過した後に出力される値。レイヤーの出力テンソルを意味し、次の
第4章

AdamW

Adam オプティマイザに重み減衰を組み合わせた最適化アルゴリズム現代LLM学習の事実上の標準。メモ
第7章

Attention (アテンション)

シーケンスの各位置が他のすべての位置にどれだけ注意を払うかを学習するメカニズム。トランスの
5章

Backpropagation

ニューラルネットワークの損失を各パラメータに微分して勾配を計算するアルゴリズム連鎖法則に基づ
第7章

BPE (Byte Pair Encoding)

バイト単位で始まり、最も頻繁に登場するペアをマージしていくトルクナイザアルゴリズム。
GPT-2、GPT-3などが使用。メモ： 第3章

Causal Mask (因果マスク)

自己回帰モデルで未来のトークンが見えないように隠すハ三角マスク。メモ： 5章

Context Length (コンテキスト長)

モデルが一度に処理できるトークンの最大数。 GPT-2は1024、GPT-4は128K。メモ： 第7章

Cross-Entropy Loss (クロスエントロピー損失)

分類問題における予測確率分布と正解分布の差を測定する損失関数言語モデリングの標準損失メモ：
第7章

Decoder (デコーダ)

トランスでシーケンスを生成する部分。 GPTはデコーダのみ使用する構造。メモ： 第6章

Dropout

学習時、いくつかのニューロンをランダムにゼロにして過適合を防ぐ正規化手法。メモ：
第4章

Embedding (埋め込み)

離散的なトークンIDを連続した密集ベクトルに変換するプロセス。またはそのベクトル自体。メモ：
第4章

Feed-Forward Network (FFN、フォワードニューラルネットワーク)

トランスブロックのアテンションの後に適用される位置別の完全接続ニューラルネットワーク。通常2
第6章

GELU (Gaussian Error Linear Unit)

ReLUのソフトバージョン。ガウス関数を使用してゼロの周りをスムーズに切り替えます。
GPTモデルが使用。メモ: 第6章

Gradient (グラデーション)

損失関数の各パラメータの微分値。最適化の方向を指します。メモ: 第7章

Greedy Decoding (貪欲デコード)

各ステップで最も確率の高いトークンのみを選択する生成戦略。高速ですが単調な出力。メモ: 第8章

ヘッド (ヘッド)

マルチヘッドアテンションで独立してアテンションを実行する単位。メモ: 5章

Layer Normalization (レイヤー正規化)

テンソルの最後の次元に沿った平均と分散を正規化する手法トランスフォーマー学習の安定化に必須。
第4章, 第6章

Learning Rate (学習率)

グラデーションのステップサイズを決定するハイパーパラメータ。大きすぎると発散、小さすぎると収束。
第7章

Logits (ロジット)

softmax を適用する前の生出力値。通常モデルの最後の線形レイヤ出力。メモ: 第7章

Loss Function (損失関数)

モデルの予測と正解の差をスカラーで測定する関数学習の最適化の対象。メモ: 第7章

Multi-Head Attention (マルチヘッドアテンション)

複数のアテンションヘッドを並列に行い、様々なパターンを同時に学習。メモ: 5章

Parameter (パラメータ)

モデルが学習する重み。線形層のW、bなど。メモ: 第7章

Perplexity (パープレクサティ、混乱度)

言語モデルの評価指標 $PPL = \exp(\text{loss})$ 。低いほど良いモデル。メモ: 第7章

Positional Encoding (位置エンコード)

変圧器に順序情報を注入する方法sin/cos または学習可能な埋め込み。メモ: 第4章

Pre-LN / Post-LN

レイヤ正規化の位置。Pre-LNはアテンション/FFNの前に、Post-LNは後に適用。Pre-LNは学習安定性が高い。
第6章

Residual Connection (残差接続)

入力を出力に加算する接続。 $y = x + f(x)$ 。深いネットワーク学習に必須。メモ: 第6章

Scaled Dot-Product Attention

$Q \cdot K^T / \sqrt{d_k}$ 形式のアテンション。トランスの最も基本的な操作。メモ: 5章

Self-Attention (自己アテンション)

同じシーケンス内のトークン間のアテンション。トランスのコア。メモ: 5章

Softmax

ベクトルを確率分布に変換する関数。 $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$ 。メモ: 5章

Temperature (温度)

サンプリング時の分布の鋭さを調整するパラメータ。低いほど決定論的、高いほどランダム。メモ：第8章

Token (トークン)

テキストを分割した最小単位。単語、部分単語、またはバイトです。メモ：第3章

Tokenizer (トークナイザー)

テキストをトークンIDシーケンスに変換し、その逆のプロセスも実行するモジュール。メモ：第3章

Top-K Sampling

次のトークン候補の中で最も確率の高いK個のみを考慮してサンプリングする戦略。メモ：第8章

Top-P (Nucleus) Sampling

累積確率がPになるまでのトークンのみを考慮するサンプリング戦略。分布形態に応じて適応的。メモ：第8章

Transformer (トランスフォーマー)

Vaswani et al. (2017) が提案したアテンションベースのニューラルネットワーク構造。現代LLMの基
第6章

Unigram Tokenizer

各トークンに確率を割り当て、EMで学習するトークナイザー。SentencePieceが使用。メモ：第3章

Viterbi Algorithm

動的計画法による最適分割を求めるアルゴリズムUnigramトークナイザーのデコードに使用。メモ：第3章

Vocabulary (語彙)

トークナイザーが認識するすべてのトークンのセット。語彙サイズは通常1万~10万。メモ：第3章

Weight Tying (ウェイトタイ)

入力埋め込みと出力層の重みを共有してパラメータ数を減らす手法メモ：第7章

ヘッダー

なぜこの本を書いたのか

2022年末にChatGPTが公開された後、大型言語モデル（LLM）は人工知能分野の最も熱いテーマになった。数
Transformersを介してすでに作成されたモデルをダウンロードし、ファインチューニングし、サービスに統
この本はその質問に答えるために使われました。トークナイザーを直接作成し、アテンションを手で実装し

この本の特徴

この本は単に理論をリストしていません。コードを最初に作成して、ユニットテストで検証した後、
その過程を本に解く。本に登場するすべてのPythonコードは実際に実行され、pytestベースのユニットテスト
各章は、前の章で実装したモジュールをインポートして使用します。たとえば、4枚の埋め込みモジュールは

誰が読むべきか

この本の主読者はPythonの基礎文法を知っていますが、ディープラーニングは初めての開発者です。
HuggingFace Transformersを書いたことがあるが、内部の動作を知らないエンジニア、「Attention
Is All You Need」の論文を読んだが、コードで実装したことはない研究者も含まれる。微積分と基礎線形代

2026年夏

dirmich

ありがとうの記事

この本を書いている間、多くのオープンソースプロジェクトと論文の助けを受けました。特にAndrej KarpathyのnanoGPT、HuggingFace Tokenizers、そしてPyTorch開発チームの優れた文書に深く感謝します。

また、この本のコードが基づいている多くの研究者に感謝します。Ashish Vaswaniとトランスフォーマー原作者Alec RadfordとOpenAIチームは、GPTシリーズを通じてスケーリングの可能性を示した。Tianyi ZhangとJason Weiは、インストラクションチューニングとソートの基礎を磨きました。

家族や友人にも感謝を伝える。執筆中に黙々と待ってくれた人々の忍耐がなければ、この本は完成しなかった。最後に、この本を読むすべての読者に。あなたの好奇心と情熱がLLMの未来を築きます。自分で作り、実験し

dirmich
Highmaru Press

Chapter 1

LLMとは何ですか

この章では、ラージ言語モデル（LLM）が何であるか、どのように進化してきたのか、そしてなぜ自分で作る

1.1 言語モデルとは

言語モデル（language model）は、一連の単語（またはトークン）に確率を割り当てる統計モデルです。簡単に
歴史的に言語モデルはN-gramモデルから始まりニューラルネットワークベースのモデル（RNN、LSTM）を経て

次のトークン予測

LLMの学習目標は単純です：これまでのトークンを見て、次のトークンを予測すること。式で書くと

$$P(w_t \mid w_1, w_2, \dots, w_{t-1})$$

この条件付き確率を正確に推定することが言語モデルの唯一の目標だ。この単純な目標が大規模なデータ
なぜそう単純な目的が強力であるか。言語を「理解する」には、文法、常識、推論、事実知識などをすべて
「地球は太陽の周りを回る…」の後に「惑星」を予測するには天文学的常識が必要であり、「1
+ 1 =」の後に「2」を予測するには算術が必要です。

1.2 直観：言語モデルをどう思うか

言語モデルに初めて触れる読者のためにたとえ話を聞いてみよう。言語モデルは「非常に多くの本を読んだ
もちろん本物の人とは違う。言語モデルは、「理解」するよりも「パターンを一致させる」。しかし、十分
4のようなモデルが驚くべき答えを出す原理だ。

言語モデルは「理解する」ですか？

この質問は哲学的な議論です。機能主義の観点からは、「入力に適切に反応すれば理解する」と見なすこ

1.3 LLMの発展の歴史

1.3.1 統計的言語モデル（1990年代 - 2000年代）

初期言語モデルは、N-gramという単純な統計技術を使用した。「直前のN-1個の単語を見て、次の単語の確率

```
1 # N-gram 言語モデルのコアアイデア簡素化()
2 from collections import Counter
3
4 corpus = "the cat sat on the mat the cat ran".split()
5 # 2-gram (bigram) カウント
```

```

6 bigrams = [(corpus[i], corpus[i+1]) for i in range(len(corpus)-1)]
7 counts = Counter(bigrams)
8 # P("mat" | "the") 推定
9 p_mat_given_the = counts[("the", "mat")] / sum(1 for w1, w2 in bigrams if w1 == "the")
10 print(f"P(mat|the) = {p_mat_given_the:.2f}")

```

しかし、N-gramには深刻な限界があった。文脈を短く見ることができ（Nが通常3~5）、未学習の組み合わせも一つの問題は、「単語の意味」を捉えられないことでした。「犬」と「犬」は同じ意味ですが、N-gram representation）に変えるニューラルネットワークベースのモデルが登場した。

1.3.2 ニューラルネットワーク言語モデル（2003 - 2017）

Bengio et al.（2003）は最初にニューラルネットワークベースの言語モデルを提案した。単語を低次元密（Dense）なベクトル（Neural Network）とLSTM（Long Short-Term Memory）が登場し、より長いコンテキストを処理できるようになった。

Word2Vecと埋め込み

Mikolovら。（2013）のWord2Vecは、単語を100~300次元ベクトルとして表現し、「似た意味の単語はベクトル差が近い」という仮定に基づき、 $\text{man} + \text{woman} \approx \text{queen}$ 。このアイデアは、第4章で実装する埋め込みの基盤です。

RNNの限界

RNNは以前の時点の隠れ状態を現在の時点に伝達する構造だ。理論的には無限のコンテキストを扱うことができるが、実際にはgradient問題が発生します。これにより、実際に処理可能なコンテキストの長さは数十~数百トークンに制限される。LSTM（Long Short-Term Memory）はゲートメカニズムでこの問題を部分的に緩和したが、根本的な解決ではなかった。100語以上の文脈はまだ難しかった。

1.3.3 トランスフォーマーの登場（2017）

2017年にVaswani et al.が発表した「Attention Is All You Need」は、言語モデルの版図を完全に変え、従来のRNNなしでアテンションメカニズムだけでシーケンスを処理するトランスフォーマー構造を提案した。トランスフォーマーの利点は以下の通り。

1. 並列処理：RNNのように順次処理せず、シーケンスのすべての位置を同時に処理することができ、GPUの並列処理能力を最大限に活用できる。
2. 長距離依存性：アテンションは、シーケンス内のすべての位置のペア間の直接接続を計算するので、距離が1000語離れた単語でも1回のアテンションで参照可能だ。

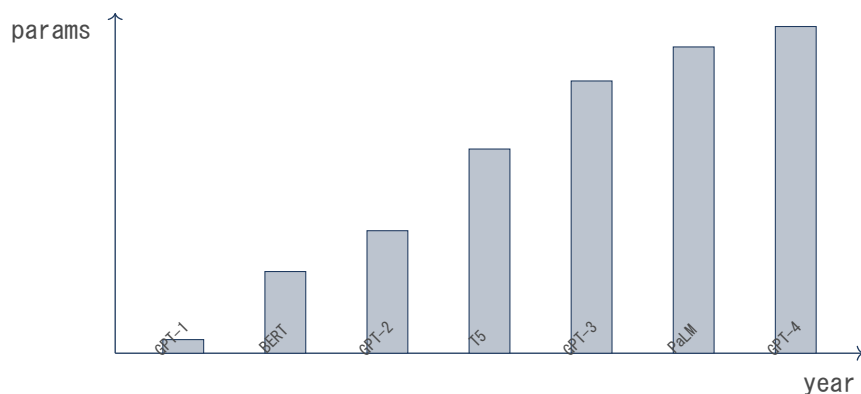


Figure 1.1: LLM パラメータ数の発展推移（ログスケール，おおよその）。GPT-1（2018）は1.1億個、GPT-4（2023）は推定1.7兆個。

1.3.4 GPT シリーズとスケーリング法則 (2018 - 現在)

OpenAIのGPTシリーズは、トランスデコーダ構造を使用して大規模なテキストで事前学習 (pre-training) するアプローチを取った。GPT-1 (2018) は1億1,700万パラメータだったが、GPT-2 (2019) は15億、GPT-3 (2020) は1,750億パラメータで急成長した。この過程でモデルサイズ、データサイズ (law) が発見された。

Kaplanら。 (2020) のスケーリング法則は、損失がモデルサイズ N 、データサイズ D 、演算量 C に対して絶

$$\mathcal{L}(N, D, C) \approx A \cdot N^{-\alpha_N} + B \cdot D^{-\alpha_D} + E \cdot C^{-\alpha_C} + L_{\infty}$$

この法則の実用的な示唆は明確です：モデルを育てるためには、データも一緒に育てなければなりません。100億パラメータモデルを10億トークンで学習すると過適合となる。Hoffmannら。 (2022) のChinchilla論

1.3.5 アライメントの登場 (2022 - 現在)

GPT-3 (2020) は1,750億のパラメータで強力だったが、ユーザーの質問に役立つ答えをしなかった。ただ「2022年にChatGPTが革新的だった理由は、モデルサイズではなくソート (alignment) にありました。

人間のフィードバックを活用した強化学習 (RLHF) がGPT-3を「役に立つチャットボット」に変換したのです。Preference Optimization (DPO)、Constitutional AIなど、よりシンプルで効率的なアライメント技術が登場。2巻8章でこれらの手法を直接実装する。

1.4 LLMの主な応用

LLMはさまざまな分野で活用されています。

1.4.1 自然言語処理の伝統的なタスク

- ・ 機械翻訳：Google Translate、DeepLなどがトランスベースです。
- ・ 要約：長い文書を短く要約。ニュース、論文、議事録など。
- ・ 感性分析：テキストの肯定/否定判別。レビュー、ソーシャルメディア分析。
- ・ オブジェクト名認識：テキストから人、場所、組織などを抽出します。

1.4.2 生成タスク

- ・ 会話：ChatGPT、Claude、Geminiなど。
- ・ コードを書く：GitHub Copilot、カーソル。Python、JavaScriptなどのさまざまな言語。
- ・ 創作：小説、詩、シナリオ。AI作家ツール。
- ・ Eメール/文書の作成：ビジネスメール、報告書草案。

1.4.3 専門分野

- ・ 医療：診断支援、医療記録の要約、患者相談。
- ・ 法律：判例の検索、契約のレビュー、法的助言。
- ・ 教育：パーソナライゼーションチューター、問題の作成、課題のスコア。
- ・ 金融：市場分析、レポート作成、詐欺の検出。

1.4.4 マルチモーダル

最新のLLMはテキストだけでなく、画像、オーディオ、ビデオも処理します。GPT-4V、Gemini、Claude 3などは、画像を理解して説明することができます。

1.5 なぜ自分で作るのか

「HuggingFaceからモデルをダウンロードすればいいのですが、なぜあなたは自分で作るのですか？」という
まず、理解の深さが異なります。model.generate()を呼び出すことと、その中で何が起こるのかを知ることは、
第二、制御力が発生します。推論速度を上げたい時、メモリを減らしたい時、新しいアーキテクチャを実験
HuggingFaceモデルをそのまま使って奇妙な出力が出たら？内部を知っている人はどこをデバッグするべきか
第三、今後のコントラストLLM分野は急速に変わります。新しい論文が毎週注がれるが、それを理解するには
RoPE、GQA、SwiGLUのような最新技術も結局アテンション、FFN、正規化の変形だ。この本を通して基本基盤
LLM-advancedで扱う分散学習、量子化、整列といった高度なテーマも自然に吸収できる。

この本の限界

この本は「実際のChatGPTを作成する」ことを目指していません。

1巻で作るモデルは1千万~1億パラメータレベルの小規模GPTで、品質はGPT-

2 small程度だ。しかし、その構造はGPT-3、GPT-4と本質的に同じである。

「サイズ」は2巻で扱われ、「構造と原理」は1巻で扱われます。

なぜ大きなモデルを作らないのですか？ 175Bモデルを学習するには、数百のGPUが数週間必要で、数億ドル

1.6 この本が作るモデル

1冊を完読すれば、読者は次のようなモデルを直接作ることになる。

- ・ アーキテクチャ：GPTスタイルのトランスデコーダ（6~12階、8ヘッド、512~768埋め込み寸法）
- ・ トークナイザー：BPE（Byte Pair Encoding）と Unigram の両方を直接実装
- ・ 学習：AdamW + cosine LRスケジュール+ウォームアップ
- ・ 作成：Greedy、Top-K、Top-P、Temperature サンプリングすべての実装
- ・ 評価：クロスエントロピーロスとパープレクティスの計算

2巻Make LLM-advancedでは、この基盤の上に分散学習（DDP、FSDP、ZeRO）、最新アーキテクチャ（RoPE、GQA、cache、PagedAttention）を追加。

1.7 学習ロードマップ

1.8 要約

- ・ 言語モデルは、テキストシーケンスに確率を割り当てるモデルであり、次のトークン予測が重要な目標
- ・ N-gramでRNN/LSTMを経て、2017年にトランスフォーマーが登場し、LLM時代が開かれた。
- ・ スケーリング法則に基づいてモデル、データ、演算を増やすと、パフォーマンスが予測可能に向上します。
- ・ 2022年以降、ソート（RLHF / DP0）は「シンプルLM」を「役に立つチャットボット」に変換しました。
- ・ 自分で作れば理解の深さ、制御力、未来対比能力が全て確保される。

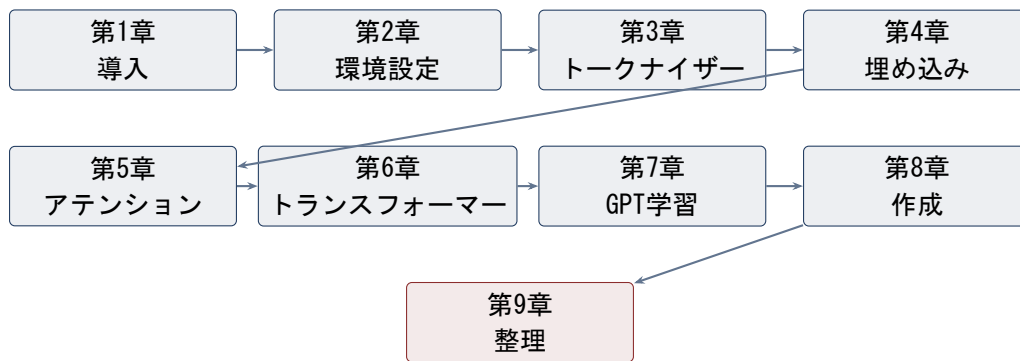


Figure 1.2: 1巻学習ロードマップ. 各章は前の章のモジュールです importで使用.

- ・ 1巻では小規模GPTを、2巻ではChatGPTレベルの技法を扱う。

1.9 練習問題

1. ChatGPT、Claude、Geminiのいずれかを使用して、次の質問に答えてください。「言語モデルが次のト
2. 次の文の次の単語の確率を人ならば、どのように結ぶかを考えてください:「太陽は東から…」。
「湧き出る」は99%以上です。なぜこの確率が高いのですか?
3. Nグラムモデルの限界のうち、「スパース性の問題」を説明してください。未学習の3グラム「ホッキョ
4. トランスがRNNよりも並列処理に有利である理由を説明してください。
5. スケーリング法則によれば、モデルを10倍増やすと、データはどれくらい増えるべきですか?
(Chinchilla基準)

次の章では、本格的な実装のためにPython、PyTorch、そしてこの本のコードストアを設定します。

Chapter 2

開発環境設定

この章では、本格的なLLM実装に先立って必要な開発環境を設定します。Python、PyTorch、そしてこの本の

2.1 必要なツール

ツール	用途	推奨バージョン
Python&実装言語		&3.10以上
PyTorch&ディープラーニングフレームワーク		&2.0以上
NumPy&数値計算		&1.24以上
pytest&ユニットテスト		&7.0以上
git&バージョン管理		&最新

GPUはオプションです。この本のすべてのコードはCPUで動作するように書かれています。ただし、大規模な2巻で分散学習を扱うときは、複数のGPU環境を想定する。

2.2 PyTorch インストール

PyTorchは公式ホームページのインストールガイドに従います。CPU専用バージョンとCUDAバージョンのうち

```
# CPU のみ (本のすべての例は として動作) CPU
pip install torch --index-url https://download.pytorch.org/whl/cpu

# CUDA 12.1 サポート (GPU 使用時)
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

設置確認は次のようにする。

```
1 import torch
2 print(torch.__version__) # 2.x.x
3 print(torch.cuda.is_available()) # GPU 利用可能かどうか
```

2.3 ストレージ構造

この本のコードは次のような構造を持っています。各章で扱うモジュールは別々のファイルに分かれており

```
make-llm-basic/
src/make_llm/
__init__.py
tokenizer/ # 第章：トークナイザー3
```

```

base.py
bpe.py
unigram.py
model/                # 第〜章：モデル構造46
    embedding.py
    attention.py
    transformer_block.py
    gpt.py
training/             # 第章：学習パイプライン7
    dataset.py
    loss.py
    optimizers.py
    trainer.py
inference/            # 第章：推論エンジン8
    sampler.py
    generate.py
utils/                # 共通ユーティリティ
    config.py
    seed.py
    logging.py
tests/                # pytest ユニットテスト
book/                 # この本の LaTeX ソース

```

2.4 最初の例：テンソルを扱う

本格的な実装に先立ち、PyTorchテンソルの基本を復習しましょう。テンソルはNumPy配列とほぼ同じように

```

1 import torch
2
3 # テンソルの作成
4 x = torch.randn(2, 3)      # 標準正規分布からのサンプリング
5 print(x.shape)             # torch.Size([2, 3])
6 print(x.dtype)             # torch.float32
7
8 # 演算
9 y = x @ x.T                # 行列積: [2, 3] @ [3, 2] -> [2, 2]
10 z = x.sum()                # すべての要素和 (スカラー)
11
12 # 自動微分
13 w = torch.randn(3, requires_grad=True)
14 loss = (w * w).sum()        # L2 norm^2
15 loss.backward()            # backward
16 print(w.grad)              # d(loss)/d(w) = 2*w

```

自動微分 (Autograd)

PyTorchの最も強力な機能の1つ。テンソルで実行されるすべての操作を内部的に「計算グラフ」として記

2.5 再現性の確保

ディープラーニング実験はランダム性が多く、再現性が重要である。シードを固定すると、同じコードを何

```

1 from makellm.utils import set_seed

```

```
2 set_seed(42) # Python, NumPy, PyTorch シードを一度に設定
```

この関数はPythonの`random`、NumPyの`numpy.random`、およびPyTorchの`torch.manual_seed`をすべて呼び出します。CUDAが使用可能な場合は、CUDAシードも設定します。

2.6 ユニットテストの実行

各章のコードは、pytestベースのユニットテストで提供されています。コードを修正したら、必ずテストを

```
cd make-llm-basic
pytest tests/ -v
```

```
===== 48 passed in 2.47s =====
```

48のテストがすべて合格すれば環境設定が完了したのだ。

2.7 要約

- ・ PyTorch 2.xとpytestをインストールしました。
- ・ リポジトリ構造を理解した: `src/make-llm/`にモジュールごとにコードが分離されている。
- ・ テンソル演算とautogradの基本を復習した。
- ・ 再現性のために`set_seed()`を使用してください。
- ・ `pytest tests/`でユニットテストを実行できます。

次の章では、本格的にトルクナイザーを実装します。テキストを数字に置き換えるこの最初の変換は、すべ

Chapter 3

トークナイザーの作成

この章では、テキストを数値シーケンスに変換するトークナイザー (tokenizer) を直接実装します。2つの主要なアルゴリズムであるBPE (Byte Pair Encoding) とUnigramを扱い、それぞれの学習、エンコーディング

3.1 なぜトークナイザーが必要なのか

ニューラルネットワークは数字だけを理解する。「Hello, world!」という文字列をそのままモデルに入れたら、反対の極端は「単語単位トークナイザー」であり、スペースで区切られた単語を1つのトークンとして扱います。of-vocabulary、OOV) を処理できません。

現代LLMは、これら2つの極端の間であるサブワード (subword) トークナイザーを使用しています。よく登場「unhappiness」は、「un」+「happiness」または「un」+「happi」+「ness」に分割することができます。

3.1.1 3つのアプローチの比較

方法	語彙サイズ	OOV 処理	シーケンス長
レタリングユニット	&小 (〜100)	&なし	&超キム
単語単位	&非常に大きい (〜100K)	&処理不能	&短い
サブワード	&中間 (〜10K〜50K)	&部分単語に分解	&中間

3.2 特殊トークン

すべてのトークナイザーはいくつかの特殊トークン (special token) を持っています。私たちの実装では4つ

トークン	ID	用途
<pad> &0 &バッチ内のシーケンスの長さを合わせるためのパディング		
<bos>	1	シーケンスの開始を表示 (Beginning Of Sequence)
<eos>	2	シーケンスの終わりを表示 (End Of Sequence)
<unk> &3 &語彙にないトークン (Unknown)		

これらのトークンはインデックスが固定されており、モデルが特別に扱うことができる。たとえば、生成時

特殊トークンID固定の理由

特殊トークンのIDを0~3に固定すると、モデルコードが単純化されます。 「if token_id == 0: mask_out」のようなハードコーディングが可能です。また、チェックポイントの互換性が保証されてい

3.3 ベースクラス

すべてのトークナイザーが従うべきインタフェースを抽象クラスとして定義します。

```
1 from abc import ABC, abstractmethod
2 from dataclasses import dataclass
3
4 @dataclass
5 class SpecialTokens:
6     pad: str = "<pad>" # ID 0
7     bos: str = "<bos>" # ID 1
8     eos: str = "<eos>" # ID 2
9     unk: str = "<unk>" # ID 3
10
11     @property
12     def tokens(self) -> list[str]:
13         return [self.pad, self.bos, self.eos, self.unk]
14
15     @property
16     def pad_id(self) -> int: return 0
17     @property
18     def bos_id(self) -> int: return 1
19     @property
20     def eos_id(self) -> int: return 2
21     @property
22     def unk_id(self) -> int: return 3
23
24
25 class Tokenizer(ABC):
26     def __init__(self, special_tokens: SpecialTokens | None = None):
27         self.special = special_tokens or SpecialTokens()
28         self.id_to_token: list[str] = list(self.special.tokens)
29         self.token_to_id: dict[str, int] = {t: i for i, t in enumerate(self.id_to_token)}
30
31     @abstractmethod
32     def _train(self, corpus: list[str], vocab_size: int) -> None:
33         """ 語彙学習 ( subclasses で実装 ) """
34
35     @abstractmethod
36     def _tokenize(self, text: str) -> list[str]:
37         """ テキストをトークンリストに分割する ( subclasses で実装 ) """
38
39     def encode(self, text: str, add_bos=False, add_eos=False) -> list[int]:
40         tokens = self._tokenize(text)
41         ids = [self.token_to_id.get(t, self.special.unk_id) for t in tokens]
42         if add_bos: ids = [self.special.bos_id] + ids
43         if add_eos: ids = ids + [self.special.eos_id]
44         return ids
45
46     def decode(self, ids: list[int]) -> str:
```

```

47     tokens = [self.id_to_token[i] for i in ids
48                 if self.id_to_token[i] not in self.special.tokens]
49     raw = self._tokens_to_bytes(tokens)
50     return self._postprocess(raw.decode("utf-8", errors="replace"))

```

抽象クラスの役割

Tokenizerは「インタフェース」を定義します。子クラス（BPE、Unigram）は`_train`と`_tokenize`を実装する必要。これが「多形性」の価値です。トレーナーは`tokenizer.encode(text)`を呼び出すだけで、内部的にBPEが回ってUnigramが回っても気にしない。後で「SentencePiece」や「WordPiece」などの新しいアルゴリズムを追加

3.4 BPE トークナイザー

BPE (Byte Pair Encoding) は、1994年にデータ圧縮のために提案されたアルゴリズムを、2016年にSennrichらGPT-2、GPT-3、LLaMAなどが使用する。

3.4.1 アルゴリズム直観

BPEのコアアイデアは単純です。

1. テキストをバイト単位で分解する（すべての文字を表現可能、00Vなし）。
2. 隣接するバイトペアの中で最も頻繁に登場するペアを探します。
3. そのペアを新しいトークンにマージします。
4. 目標語彙サイズに達するまで2~3を繰り返します。

最初はすべてのテキストが「バイトのリスト」です。たとえば、「ローワー」は`[l, o, w, e, r]`で表されます。

- ・ 「lo」 が頻繁に登場すると、マージ：`[lo, w, e, r]`
- ・ 「er」 が頻繁に登場すると、マージ：`[lo, w, er]`
- ・ 「low」 が頻繁に登場すると、マージ：`[low, er]`
- ・ 「Lower」 が頻繁に登場すると、マージ：`[lower]`

最後に、「ローワー」は1つのトークンになりますが、まれな単語は部分単語として残ります。「Lowering」は「Lower」+「ing」になります。

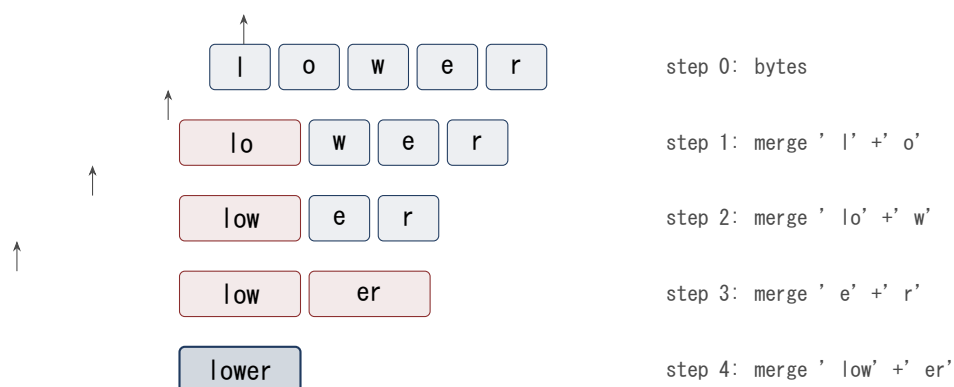


Figure 3.1: BPE 学習コース. “lower” がバイトで始まり、徐々にマージされて1つのトークンになるプロセス.

3.4.2 なぜバイト単位なのか？

GPT-2以降、BPEは「バイト単位」で動作します。以前は文字単位を書きましたが、問題がありました。

- ・ 絵文字、珍しい漢字など語彙にない文字は<unk>処理されます。
- ・ 多言語コーパスで文字セットが大きすぎます。

バイト単位の場合、すべてのUTF-8文字を1 - 0 4バイトで表現でき、語彙の「バイト」部分は常に256に固定00Vは事実上消えます。

3.4.3 実装：学習

```
1 from collections import Counter
2 from .base import Tokenizer
3
4 class BPETokenizer(Tokenizer):
5     def __init__(self, special_tokens=None):
6         super().__init__(special_tokens=special_tokens)
7         self.merges: list[tuple[str, str]] = []
8         self._merge_rank: dict[tuple[str, str], int] = {}
9
10    def _train(self, corpus: list[str], vocab_size: int) -> None:
11        # 1. 特殊トークン + バイトで初期語彙を構成する256
12        self.id_to_token = list(self.special_tokens)
13        for b in range(256):
14            self.id_to_token.append(chr(b))
15        self.token_to_id = {t: i for i, t in enumerate(self.id_to_token)}
16
17        # 2. コーパスを単語単位に分割
18        word_freqs = Counter()
19        for text in corpus:
20            for word in self._pre_tokenize(text):
21                word_freqs[word] += 1
22
23        # 3. 各単語をバイトトークンリストに変換
24        word_to_tokens = {
25            w: [chr(b) for b in w.encode("utf-8")]
26            for w in word_freqs
27        }
28
29        # 4. マージループ
30        self.merges = []
31        target = vocab_size - len(self.id_to_token)
32        for _ in range(max(target, 0)):
33            pair_freqs = Counter()
34            for word, freq in word_freqs.items():
35                toks = word_to_tokens[word]
36                for i in range(len(toks) - 1):
37                    pair_freqs[(toks[i], toks[i+1])] += freq
38            if not pair_freqs:
39                break
40            best = max(pair_freqs.items(), key=lambda x: (x[1], x[0]))
41            if best[1] < 2:
42                break
43            new_token = best[0][0] + best[0][1]
44            self.merges.append(best[0])
```



```

45     self.id_to_token.append(new_token)
46     self.token_to_id[new_token] = len(self.id_to_token) - 1
47     for word in word_to_tokens:
48         word_to_tokens[word] = self._merge_in_list(
49             word_to_tokens[word], best[0], new_token
50         )
51
52     self._merge_rank = {p: i for i, p in enumerate(self.merges)}

```

マージ優先順位の安定性

`max(pair_freqs.items(), key=lambda x: (x[1], x[0]))` から `(x[1], x[0])` タプルにソートする理由は、同点処理によるもの

3.4.4 エンコーディング

学習したマージルールを新しいテキストに適用するには、最も優先順位の高い（最初に学習した）マージか

```

1     def _tokenize(self, text: str) -> list[str]:
2         tokens = []
3         for word in self._pre_tokenize(text):
4             bs = word.encode("utf-8")
5             word_tokens = [chr(b) for b in bs]
6             word_tokens = self._apply_merges(word_tokens)
7             tokens.extend(word_tokens)
8         return tokens
9
10    def _apply_merges(self, tokens: list[str]) -> list[str]:
11        """学習したマージ規則を優先順位に適用する."""
12        while len(tokens) >= 2:
13            # 最も低い（優先順位が高い）ペアを探すrank
14            best_idx, best_rank = -1, len(self.merges) + 1
15            for i in range(len(tokens) - 1):
16                rank = self._merge_rank.get((tokens[i], tokens[i+1]), -1)
17                if 0 <= rank < best_rank:
18                    best_rank, best_idx = rank, i
19            if best_idx < 0:
20                break
21            merged = tokens[best_idx] + tokens[best_idx + 1]
22            tokens = tokens[:best_idx] + [merged] + tokens[best_idx + 2:]
23        return tokens

```

3.4.5 前処理：スペース処理

BPEはスペースを特別に扱わなければ単語の境界を保存できない。GPT-2はスペースを特殊文字`Ġ`（U + 0120）に置き換えます。私たちはSentencePieceスキームに従ってU + 2581（`▯`）に置き換えます。

```

1     def _pre_tokenize(self, text: str) -> list[str]:
2         text = text.replace(" ", "▯")
3         text = text.replace("\n", "▯").replace("\t", "▯")
4         while "▯" in text:
5             text = text.replace("▯", " ")
6         if not text.startswith("▯"):
7             text = "▯" + text
8         parts = text.split("▯")
9         return ["▯" + p for i, p in enumerate(parts) if i > 0 and p]

```

空白置換の罠

「Hello world」を「Hello world」に置き換えてからスプリットすると["_Hello", "_world"]になります。単語ごとにこの前に付いており、デコード時を再び空白に変えると元の文章が復元されます。しかし、「Hello、world!」のように句読点がある場合は注意が必要です。単純なスプリットは「Hello」2は正規表現パターンで単語/句読点を分離しますが、この本では単純化のために省略します。

3.4.6 デコード：バイトの復元

BPEのトリッキーな部分はデコードです。トークンは`chr(byte)`のシーケンスなので、各文字のコードポイント8にデコードする必要があります。

```
1 def _tokens_to_bytes(self, tokens: list[str]) -> bytes:
2     """BPE トークンをバイトに変換。各文字のコードポイントをバイト単位で."""
3     out = bytearray()
4     for tok in tokens:
5         for c in tok:
6             out.append(ord(c) & 0xFF)
7     return bytes(out)
```

たとえば、「Hello」トークンがある場合、(U + 2581) はUTF-8で0xE2 0x96 0x81なので3バイトですが、BPE学 + `chr(0x96) + chr(0x81)` で表されます。デコード時に各`chr(b)` から`ord()` にバイトを抽出すると元のバイトストリー 0x96 0x81 0x48 0x65 0x6C 0x6C 0x6F が得られ、それをUTF-8にデコードすると「Hello」になります。

3.5 Unigram トークナイザー

Unigramトルクナイザーは、Kudo (2018) がSentencePieceの一部として提案した方法です。BPEが「頻度ベ

3.5.1 直観

Unigramは各トークンに確率を割り当てます。文章「ローワー」について考えられる分割はいくつかあります。

- ・ [l, o, w, e, r] : 5つのトークン
- ・ [lo, w, er] : 3つのトークン
- ・ [low, er] : 2つのトークン
- ・ [lower] : 1トークン

各分割の確率はトークン確率の積として計算される。最も高い確率を持つ分割を選択します。

「低」が語彙にあり、確率が高い場合は[lower]が選択されますが、存在しない場合は小さい単位に分割され

3.5.2 アルゴリズム

1. コーパスからすべての部分文字列を候補トークンに抽出し、頻度を数えます。
2. 頻度を確率に変換して、各トークンにログ確率スコアを割り当てます。
3. EM (Expectation-Maximization) 反復 :
 - ・ E-step : Viterbiアルゴリズムによる各単語の最適分割ナビゲーション。
 - ・ M-step: 分割結果でトークン確率を更新。

4. 最もスコアの低いトークンを削除し、語彙を縮小します。

3.5.3 Viterbi デコード

与えられた単語に対して「最も高い総ログ確率を持つトークン分割」を動的計画法で見つけます。

```
1 def _viterbi(self, word: str) -> tuple[list[str], float]:
2     n = len(word)
3     # dp[i] = 最大スコア(, 前のインデックス, 以前のトークン)
4     dp = [(-float("inf"), -1, "") for _ in range(n + 1)]
5     dp[0] = (0.0, -1, "")
6     for i in range(1, n + 1):
7         for j in range(max(0, i - 16), i):
8             substr = word[j:i]
9             score = self.token_scores.get(substr, -20.0)
10            cand = dp[j][0] + score
11            if cand > dp[i][0]:
12                dp[i] = (cand, j, substr)
13
14            # 逆追跡
15            path, i = [], n
16            while i > 0:
17                _, j, tok = dp[i]
18                path.append(tok)
19                i = j
20            path.reverse()
21            return path, dp[n][0]
```

Viterbiの複雑さ

$O(n^2)$ の代わりに $O(n \cdot L)$ に縮小された秘密は $\max(0, i-16)$ 部分です。トークンの長さが16を超えることはまれ

3.6 BPE vs Unigram 比較

	BPE	Unigram
学習方法&頻度ベースのマージ&確率ベースの分割 (EM)		
エンコード&マージルールの順序を適用&Viterbiで最適な分割		
決定論的&0 (同じ入力、同じ出力) &0		
マルチスプリットサポート	X	0 (サンプリング可能)
主な用途	GPT-2, GPT-3, LLaMA	T5, AL- BERT, mBART

3.7 トルクナイザーテスト

実装が終わったら、単体テストで検証します。最も重要なテストは、ラウンドトリップ (round-trip) テスト: エンコード後にデコードすると原文を復元する必要があるということです。

```
1 def test_encode_decode_roundtrip():
2     tok = BPETokenizer()
3     tok.train(CORPUS, vocab_size=200)
4     text = "the quick fox"
```

```

5     ids = tok.encode(text)
6     decoded = tok.decode(ids)
7     assert " ".join(decoded.split()) == "the quick fox"
8
9 def test_bos_eos_addition():
10     tok = BPETokenizer()
11     tok.train(CORPUS, vocab_size=100)
12     ids = tok.encode("hello", add_bos=True, add_eos=True)
13     assert ids[0] == tok.special.bos_id
14     assert ids[-1] == tok.special.eos_id
15
16 def test_unknown_token_handled():
17     """語彙にない文字もバイト単位で処理されます。が発生しないことunk."""
18     tok = BPETokenizer()
19     tok.train(CORPUS, vocab_size=100)
20     ids = tok.encode("こんにちは")
21     assert tok.special.unk_id not in ids # バイトに分解されるので unk なし
22
23 def test_save_load(tmp_path):
24     tok = BPETokenizer()
25     tok.train(CORPUS, vocab_size=150)
26     path = tmp_path / "tok.json"
27     tok.save(path)
28     tok2 = BPETokenizer.load(path)
29     text = "the quick fox"
30     assert tok.encode(text) == tok2.encode(text)

```

テスト哲学

この本のすべてのモジュールには3つのレベルのテストがあります。

- ・ 形状テスト (shape test) : テンソルの形状が予想と一致するかどうか。
- ・ 数値テスト (numeric test) : 小さい入力に対する手計算結果と比較。
- ・ 統合テスト : 複数のモジュールを接続したときのエンドツーエンドの動作。

これら3つを通過したコードだけが本に収録される。

ラウンドトリップテストは統合テストの一例です。「エンコード後にデコードすると原文が出る」という

3.8 練習： 実際の馬でトークナイザーを学ぶ

本のscripts/tiny_train.pyデモを実行すると、実際にBPEトークナイザーを学習するプロセスを見ることができま

```

cd make-llm-basic
python scripts/tiny_train.py --vocab 500

```

出力例 :

```

1 [1/5] トークナイザー学習中...
2   完了: トークン308, 秒1.2
3   サンプル: 'the quick brown fox jumps over the lazy dog'
4   → トークン18: [264, 282, 291, 267, 261, 110, ...]

```

3.9 要約

- ・ トルクナイザーは、テキストを整数シーケンスに変換するLLMの最初のステップです。
- ・ サブワード方式は語彙サイズとOOVのバランスをとります。
- ・ BPEは頻度ベースのマージ、Unigramは確率ベースの分割。
- ・ 特殊トークン<pad>/<bos>/<eos>/<unk>はID 0~3に固定されています。
- ・ バイト単位のBPEはOOVを事実上除去します。
- ・ ViterbiアルゴリズムでUnigramの最適分割を $O(n)$ に探します。
- ・ すべての実装はpytestユニットテストで検証されています。

3.10 練習問題

1. BPEを学習するとき、「最も頻度の高いペア」の代わりに「最も頻度の低いペア」をマージするとどうなるか？
2. Unigramトルクナイザーで`score = -20.0`ペナルティ値を`-5.0`に変更するとどうなりますか？
3. 韓国語コーパス「こんにちはお会いできて嬉しいです」をBPEとして学びましょう。語彙サイズが1000以下になるか？
4. Viterbiでトークン長の上限を16ではなく4に減らすと、どのトークンが消えますか？
5. <bos>トークンがなぜ必要なのか考えてください。なくてもモデルが動作するか？

次の章では、トークンIDを密集ベクトルに変換する埋め込みレイヤを実装します。

Chapter 4

埋め込みとポジショナルエンコーディング

この章では、トークンIDを密集ベクトルに変換する埋め込み (embedding) と、シーケンスの順序情報をモデルに伝えるエンコーディング (encoding) を実装します。

4.1 なぜ埋め込みが必要なのか

第3章では、トークナイザーが「Hello」を整数ID 247に変更しました。しかし、この定数をそのままニューラルネットワークに入力すると、「cat」は247、「dog」は248、または「zebra」は248です。第二に、整数を乗算または加算する演算は意味が通じません。「cat + 1 = dog」は成立しません。

解決策は、各トークンIDを密集ベクトルに変換することです。語彙サイズが V で埋め込み次元が d の場合、埋め込み行列 $E \in \mathbb{R}^{V \times d}$ の学習可能な行列 E です。トークンID i が入ると、 E の i 行目が取得されます。この行ベクトルがそのトークンの埋め込みです。

なぜ「密集」なのか？

ワンホットエンコーディングは、語彙サイズ V のスパースベクトルでトークンを表現します。たとえば、ID 247は246個のゼロと1個の1、残りの0からなるベクトルです。これを E と乗算すると、最終的に E の247番目の行が取得されます。学習が進むと、同様の意味の単語がベクトル空間に近づきます。

「cat」と「dog」は近く、「cat」と「car」は遠いです。これが埋め込みの魔法である：学習前はランダムベクトルであったが、学習後には意味を込めた座標となる。

4.2 埋め込みの数学的定義

埋め込み行列 $E \in \mathbb{R}^{V \times d}$ がある場合、トークンID i の埋め込みは $E[i] \in \mathbb{R}^d$ です。これは単に E の i 番目の行を選択 (backward 時選択された行だけグラデーションを受け取る)。

バッチ入力 $x \in \mathbb{Z}^{B \times L}$ (バッチ B 、シーケンス長 L) の場合、埋め込み出力は $X \in \mathbb{R}^{B \times L \times d}$ になります。

4.3 Token Embedding 実装

PyTorchの`nn.Embedding`は正確にこのlookup操作を実行します。私たちはそれを薄くラップして初期化ロジックを簡潔にします。

```
1 import math
2 import torch.nn as nn
3
4 class TokenEmbedding(nn.Module):
5     def __init__(self, vocab_size: int, d_model: int, pad_id: int = 0):
6         super().__init__()
7         self.vocab_size = vocab_size
8         self.d_model = d_model
9         self.pad_id = pad_id
```

```

10 self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_id)
11 self._init_weights()
12
13 def _init_weights(self):
14     # 正規分布  $\mathcal{N}(0, 1/\sqrt{d\_model})$  の初期化 (GPTスタイル-2)
15     nn.init.normal_(self.embedding.weight, mean=0.0,
16                     std=1.0 / math.sqrt(self.d_model))
17     # パディングトークンは0
18     with torch.no_grad():
19         self.embedding.weight[self.pad_id].fill_(0.0)
20
21 def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
22     # [batch, seq] -> [batch, seq, d_model]
23     return self.embedding(token_ids)

```

初期化の重要性

ディープラーニングにおける重みの初期化は学習成否に左右される。大きすぎるとアクティベーション値が爆発し、小さすぎると消失する。GPT-2は標準偏差 $1/\sqrt{d_model}$ の正規分布を使用し、これは初期埋め込みベクトルのnormが約1になるような値である。なぜあなたは $1/\sqrt{d}$ ですか？ d 次元ベクトルの各要素が $\mathcal{N}(0, \sigma^2)$ の場合、ベクトルnormの期待値は $\sqrt{d} \cdot \sigma$ です。 $\sigma = 1/\sqrt{d}$ なら norm が 1 近くなる。これにより、埋め込みベクトルのスケールがレイヤ出力に伝わる。

4.3.1 padding_idxの役割

nn.Embeddingのpadding_idxパラメータは、パディングトークンの埋め込みを常にゼロに保ちます。これにより：

- ・ パディング位置の出力が0になり、アテンション計算に影響を与えません。
- ・ backward 時、パディング行のグラデーションが 0 になり、重みが更新されない。

4.4 ポジショナルエンコーディング

トランスはアテンションだけでシーケンスを処理しますが、アテンション自体には「順序」という概念はあ (invariance) という。自然言語では、順序が意味を決定するので（「犬が人を尋ねた」対「人が犬を尋ねた」）。

4.4.1 直観：なぜ順序が重要なのか？

アテンションは、「各トークンが他のトークンにどれだけ注意を払うか」を学びます。しかし、「最初のトークン」は、各位置に固有の「位置ベクトル」を追加し、モデルが位置を区別できる。

4.4.2 Sinusoidal 位置エンコーディング (Vaswani et al. 2017)

オリジナルトランスフォーマー論文が提案した方式。サインとコサイン関数で位置をエンコードします。

Sinusoidal Positional Encoding

$$PE_{\text{pos}}(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_model}}\right)$$

$$PE_{\text{pos}}(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{2i/d_model}}\right)$$

ここで、 pos は位置、 i はディメンションインデックスです。偶数次元はsin、奇数次元はcos。

この方式の利点は、学習可能なパラメータがないことです。また、相対位置に対する線形関係が成立し、モ

$\beta) = \sin \alpha \cos \beta + \cos \alpha \sin \beta$ なので、位置 $pos+k$ の符号化は、位置 pos の符号化の線形変換として表される。

```

1 class PositionalEmbedding(nn.Module):
2     def __init__(self, d_model: int, max_len: int = 512, mode: str = "learned"):
3         super().__init__()
4         self.mode = mode
5         if mode == "sinusoidal":
6             pe = torch.zeros(max_len, d_model)
7             position = torch.arange(0, max_len).unsqueeze(1).float()
8             div_term = torch.exp(
9                 torch.arange(0, d_model, 2).float() * (-math.log(10000.0) / d_model)
10            )
11             pe[:, 0::2] = torch.sin(position * div_term)
12             pe[:, 1::2] = torch.cos(position * div_term)
13             self.register_buffer("pe", pe.unsqueeze(0)) # [1, max_len, d_model]
14         else: # learned
15             self.pe = nn.Embedding(max_len, d_model)
16             nn.init.normal_(self.pe.weight, mean=0.0, std=1.0/math.sqrt(d_model))
17
18     def forward(self, seq_len: int) -> torch.Tensor:
19         # [1, seq_len, d_model] 返す
20         if seq_len > self.max_len:
21             raise ValueError(f"Sequence length {seq_len} exceeds max_len {self.max_len}")
22         if self.mode == "sinusoidal":
23             return self.pe[:, :seq_len, :]
24         else:
25             positions = torch.arange(seq_len, device=self.pe.weight.device)
26             return self.pe(positions).unsqueeze(0)

```

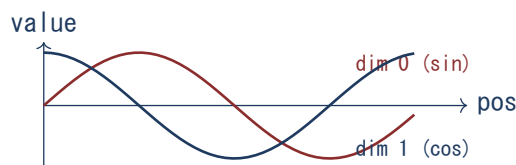


Figure 4.1: Sinusoidal 位置エンコーディングの最初の2次元. sinとcosこの位置によって周期的に変化する.

4.4.3 学習可能な位置埋め込み (GPT-2スタイル)

GPT-2、GPT-3などは、位置ごとに学習可能な埋め込みベクトルを使用する。このアプローチは、

Sinusoidal vs Learned

- ・ Sinusoidal : パラメータなし、extrapolation可能（学習時の本長より長いシーケンスにも対応）。
- ・ Learned : パフォーマンスは少し優れていますが、固定長。extrapolationの難しさ。

最新のモデルは、RoPE (Rotary Position Embedding) を使用して2つの方法の利点を組み合わせています。2巻5章で扱う。

4.5 埋め込み合成

トークンの埋め込みと位置の埋め込みを加えて、最終的な入力表現を作成します。Vaswani et al. は、埋め込みに $\sqrt{d_{\text{model}}}$ を掛けてスケールを調整することを提案しました。これは、位置埋め込みと

```
1 class EmbeddingStack(nn.Module):
2     def __init__(self, vocab_size, d_model, max_len=512,
3                   pad_id=0, pos_mode="learned", dropout=0.1):
4         super().__init__()
5         self.token_emb = TokenEmbedding(vocab_size, d_model, pad_id=pad_id)
6         self.pos_emb = PositionalEmbedding(d_model, max_len, mode=pos_mode)
7         self.dropout = nn.Dropout(dropout)
8         self.scale = math.sqrt(d_model)
9
10    def forward(self, token_ids: torch.Tensor) -> torch.Tensor:
11        batch, seq = token_ids.shape
12        tok = self.token_emb(token_ids) * self.scale # スケーリング
13        pos = self.pos_emb(seq)
14        return self.dropout(tok + pos)
```

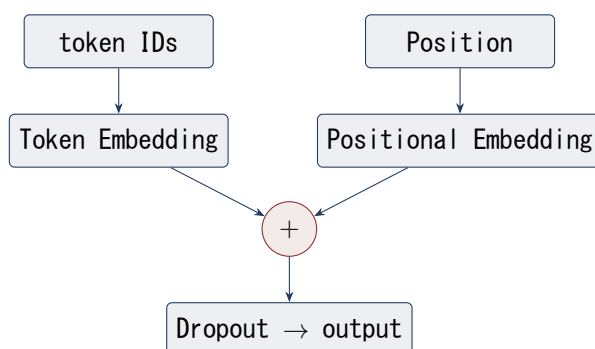


Figure 4.2: EmbeddingStack 構造. トークン埋め込みと位置埋め込みを追加した後にドロップアウトを適用する.

4.5.1 なぜスケールが必要なのか?

トークン埋め込みは $\mathcal{N}(0, 1/d)$ に初期化されており、元素の分散が $1/d$ です。位置埋め込みの元素は $[-1, 1]$ の範囲にあり、分散が $1/3$ です。位置埋め込みの分散が $1/d$ になるように $\sqrt{d}$ を掛けて、トークン埋め込みの分散が 1 になり、位置埋め込みとバランスが取れます。

4.6 Layer Normalization はじめに

トランスで学習を安定させるためにレイヤー正規化 (Layer Normalization) を使用してください。次章のアーキテクチャ

Layer Normalization

$$\text{LN}(x) = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

ここで、 μ, σ^2 は最後の次元に沿って計算された平均と分散です。 γ, β は学習可能なパラメータです。

PyTorchでは `nn.LayerNorm(d_model)` を1行で使用します。2巻ではLayerNormの変形であるRMSNormも扱う。

LayerNorm vs BatchNorm

BatchNormは配置次元に沿って正規化し、学習と推論の際に統計が異なる問題がある。

LayerNormは各サンプル内で正規化し、バッチサイズに関係なく動作します。これがLLMでLayerNormを書くバッチサイズ1の推論でも安定して動作する。

4.7 テスト

埋め込みモジュールのコアテストは、出力形状の確認と勾配の流れの確認です。

```
1 def test_token_embedding_shape():
2     emb = TokenEmbedding(vocab_size=100, d_model=32)
3     ids = torch.randint(0, 100, (2, 8)) # [batch=2, seq=8]
4     out = emb(ids)
5     assert out.shape == (2, 8, 32)
6
7 def test_positional_sinusoidal():
8     pe = PositionalEmbedding(d_model=32, max_len=64, mode="sinusoidal")
9     out = pe(seq_len=16)
10    assert out.shape == (1, 16, 32)
11
12 def test_embedding_stack():
13     stack = EmbeddingStack(vocab_size=100, d_model=32, max_len=64, dropout=0.0)
14     ids = torch.randint(0, 100, (2, 8))
15     out = stack(ids)
16     assert out.shape == (2, 8, 32)
17
18 def test_positional_exceeds_max_len_raises():
19     pe = PositionalEmbedding(d_model=32, max_len=32, mode="learned")
20     with pytest.raises(ValueError):
21         pe(seq_len=64) # max_len 超過
```

4.8 練習：埋め込み可視化

scripts/tiny_train.pyで学習された埋め込みをPCAで2次元に投影すると、単語がどのように分布するかがわかり

4.9 要約

- ・ 埋め込みは、トークンIDを密集ベクトルに変換する学習可能なlookupテーブルです。
- ・ 初期化は $N(0, 1/\sqrt{d_{\text{model}}})$ を使用し、パディングトークンは0に設定します。
- ・ ポジショナルエンコーディングはトランスに順序情報を注入します。sinusoidalとlearnedの2つの方法
- ・ トークン埋め込みと位置埋め込みを追加し、ドロップアウトを適用して最終入力表現を作成します。
- ・ $\sqrt{d_{\text{model}}}$ スケーリングで2つの埋め込みの分散をバランスさせます。
- ・ LayerNormはバッチサイズに依存しない正規化で、LLM学習の安定化に不可欠です。

4.10 練習問題

1. 埋め込みディメンション d が小さすぎる場合（例：8）、どのような問題が発生しますか？大きすぎる場

2. Sinusoidal位置エンコーディングで10000という定数を100に変更するとどうなりますか？
 3. トークン埋め込みに $\sqrt{d_{model}}$ を掛けないと、学習はどのように異なりますか？
 4. 学習可能な位置埋め込みの`max_len`を1024に設定して学習した後、推論時に2048の長さのシーケンスを入力するとどうなりますか？
 5. LayerNormの ϵ 値を 10^{-5} から 10^{-1} に変更すると、出力がどのように変わりますか？
- 次の章では、この埋め込みを入力として受け取り、「各トークンが他のトークンにどれだけ注意を払うか」

Chapter 5

アテンション機構

この章では、トランスの重要なアテンション (attention) メカニズムを実装します。 Scaled Dot-Product Attentionから始めて、Multi-Head Attentionまで段階的に見ていきます。

5.1 直観: アテンションとは何ですか

文章「マット・ベカウス・ザ・キャット・サットが00 - はタイト」で「it」が何を指していますか？人は簡式で、アテンションは、与えられたクエリ Q に対して、すべてのキー K との類似度を計算し、その類似度で付

アテンションの一般定義

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_k}} \right) \cdot V$$

- ・ Q : クエリ (今見ているトークンの表現)
- ・ K : キー (他のトークンの「ラベル」)
- ・ V : 値 (他のトークンの「内容」)
- ・ d_k : キーの次元。 $\sqrt{d_k}$ で割る理由は dot product が大きくなり softmax が飽和するのを防ぐためです。

5.1.1 直感的なたとえ話: 図書館検索

図書館で本を探す状況を想像しましょう。あなたが望む情報 (クエリ Q) を持って図書館に行きます。各本のラベルはキー K 、内容は値 V です。アテンションはこの過程を微分可能にしたものだ。「類似度」をドット製品に、「重み付き合計」をsoftmax

5.2 Q, K, V どこから来るの？

トランスでは、 Q, K, V はすべて同じ入力 x の線形変換で作成されます。

$$Q = xW_Q, \quad K = xW_K, \quad V = xW_V$$

ここで W_Q, W_K, W_V は学習可能な行列です。これにより、同じ入力について「質問」、「ラベル」、「

なぜ3つの異なる行列ですか？

同じ x を3回使用せずに W_Q, W_K, W_V に変換するのは、「役割分担」によるものです。クエリは「何が

5.3 スケーリング：なぜ $\frac{1}{\sqrt{d_k}}$ に分けるか？

$Q \cdot K^T$ は d_k 次元ベクトルの内積です。 d_k が大きいと内積値が大きくなります。 Q, K の元素が平均0、分散1の内積値が大きいと、softmax出力はワンホットに近づきます（1つの値は1、残りの0）。こうなるとグラデー

スケーリングの効果

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \quad \text{vs} \quad \text{softmax}(QK^T)$$

スケーリングしないと $d_k = 64$ のとき内積値が ± 64 まで行くことができ、softmaxが飽和。スケーリングす

5.4 Scaled Dot-Product Attention 実装

最も基本的な形から始めましょう。PyTorch 2.0の `F.scaled_dot_product_attention` (SDPA) を使用すると、メモリ

```
1 import torch
2 import torch.nn.functional as F
3
4 class ScaledDotProductAttention(nn.Module):
5     def __init__(self, dropout: float = 0.0):
6         super().__init__()
7         self.dropout_p = dropout
8
9     def forward(self, q, k, v, mask=None):
10         # q, k, v: [batch, n_heads, seq, d_head]
11         # mask: [batch, 1, seq, seq] または [seq, seq]
12         out = F.scaled_dot_product_attention(
13             q, k, v, attn_mask=mask,
14             dropout_p=self.dropout_p if self.training else 0.0,
15         )
16         return out
```

SDPAはC++レベルで最適化されており、Pythonループでsoftmaxを直接実装するよりも数十倍速い。また、Flash Attentionのような最新のアルゴリズムを自動的に活用する。

Flash Attention

Flash Attention (Dao et al. 2022) は、アテンション計算をGPUメモリ階層に合わせて再設計し、メモリPyTorch 2.0のSDPAは自動的にFlash Attentionを有効にします。私たちが直接実装する必要なしに `F.scaled_dot_product_attention`

5.5 因果マスク (Causal Mask)

自己回帰言語モデルでは、「未来」を見るべきではありません。トークン t を予測するときにトークン $t+1, t+2, \dots$ を見てはいけないという意味だ。このためにアテンションスコア行列の上三角（主対角線上）を $-\infty$ softmax 後 0 になるようにする。

```
1 def make_causal_mask(seq_len: int, device="cpu") -> torch.Tensor:
2     """ハ三角マスク。将来のトークンでマスキングinf."""
3     mask = torch.full((seq_len, seq_len), float("-inf"), device=device)
4     mask = torch.triu(mask, diagonal=1) # 主対角線の上 -inf
5     return mask
```

		token 0			token 4	
token 0	0	-ir	-ir	-ir	-inf	
	0	0	-ir	-ir	-inf	
	0	0	0	-ir	-inf	
	0	0	0	0	-inf	
token 4	0	0	0	0	0	

Figure 5.1: 因果マスク (5 - 5). 青 = アテンション許可(0), 赤 = 将来のトークンをブロックする (-1 -). 行 - 2 - はトークン - 3 - が熱心 - 4 - トークンにどれだけ注意を払うかを示す.

マスクをアテンションスコアに加えると、マスクされた位置のスコアは $-\infty$ になり、softmaxの後は0になります。

5.5.1 なぜ - 認可?

softmaxの式 $\text{softmax}(x_i) = e^{x_i} / \sum_j e^{x_j}$ から $x_i = -\infty$ なら $e^{-\infty} = 0$ になります。したがって、マスクされた

5.6 Multi-Head Attention

単一のアテンションは1つの「視点」のみを学習できます。しかし、言語にはさまざまな関係がある：文法的（動詞）、意味的關係（同義語、反義語）、談話関係（指語参照解決）など。マルチヘッドアテンション（Multi-Head Attention）はこれを複数のヘッドに分割して並列処理します。

5.6.1 アイデア

d_{model} 次元を h 個のヘッドに分割し、各ヘッドが $d_{\text{head}} = d_{\text{model}}/h$ 次元で独立してアテンションを行う。
- 3 - の場合は $d_{\text{head}} = 64$ 。

Multi-Head Attention

$$\text{MHA}(x) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W_O$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$

各ヘッドの出力をつなぎ合わせた後、 W_O に線形変換。

5.6.2 なぜ頭を分けるのか?

$d_{\text{model}} = 512$ の単一アテンションと $d_{\text{head}} = 64, h = 8$ のマルチヘッドアテンションは、合計パラメータ数は

- ・ ヘッド1：主語と動詞の関係
- ・ ヘッド2：形容詞 - 名詞式
- ・ ヘッド3：ディレクティブリファレンス
- ・ ヘッド4：文章境界

実際にアテンションヘッドを視覚化してみると（例えばBertViz）、異なるヘッドが確かに異なるパターンに

5.6.3 実装

効率のために Q, K, V を一度に計算します。入力 x に対して $3d_{\text{model}}$ 次元に投影した後、ヘッド別に分割する

```
1 class MultiHeadAttention(nn.Module):
```

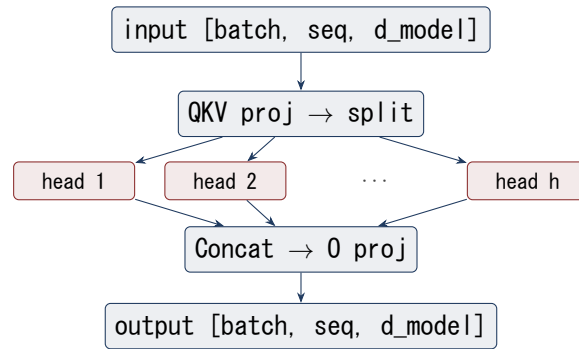


Figure 5.2: Multi-Head Attention 構造. 入力 - 犬ヘッドに分割して並列アテンションを行った後の合針.

```

2  def __init__(self, config: ModelConfig):
3      super().__init__()
4      self.d_model = config.d_model
5      self.n_heads = config.n_heads
6      self.d_head = config.d_model // config.n_heads
7      # Q, K, を一度に計算 (結合プロジェクト) V
8      self.qkv_proj = nn.Linear(config.d_model, 3 * config.d_model, bias=config.use_bias)
9      self.o_proj = nn.Linear(config.d_model, config.d_model, bias=config.use_bias)
10     self.attention = ScaledDotProductAttention(dropout=config.dropout)
11     self.dropout = nn.Dropout(config.dropout)
12
13     def forward(self, x, mask=None):
14         batch, seq, _ = x.shape
15         # [batch, seq, 3*d_model] -> [batch, seq, 3, n_heads, d_head]
16         qkv = self.qkv_proj(x).view(batch, seq, 3, self.n_heads, self.d_head)
17         # [3, batch, n_heads, seq, d_head]
18         qkv = qkv.permute(2, 0, 3, 1, 4)
19         q, k, v = qkv[0], qkv[1], qkv[2]
20
21         if mask is not None and mask.dim() == 2:
22             mask = mask.unsqueeze(0).unsqueeze(0) # [1, 1, seq, seq]
23
24         # アテンション: [batch, n_heads, seq, d_head]
25         attn_out = self.attention(q, k, v, mask=mask)
26         # ヘッドを合わせる: [batch, seq, d_model]
27         attn_out = attn_out.transpose(1, 2).contiguous().view(batch, seq, self.d_model)
28         return self.dropout(self.o_proj(attn_out))

```

なぜQKVを一度に計算するのですか？

3つのnn.Linearを別々に書くのではなく、3 * d_model出力を持つ1つのnn.Linearを書く理由は効率性です。GPUは、大きな行列演算を一度に実行するために最適化されています。3回の小さな演算よりも1回の大きな演算が高速です。分割はview + permuteでメモリ再構成なしで実行されます。

5.7 因果性テスト

アテンションモジュールの最も重要なテストは、「因果マスクが実際に将来のトークンをブロックするか」

```

1 def test_attention_is_causal(small_config):
2     """場所 i 場所 i以降の影響を受けないでください+1."""
3     attn = MultiHeadAttention(small_config)
4     attn.eval()
5     x = torch.randn(1, 6, small_config.d_model)
6     mask = make_causal_mask(6)
7     out1 = attn(x, mask=mask)
8     # の最後の位置を変えても前位置の出力は変わらないはずx
9     x2 = x.clone()
10    x2[:, -1] = torch.randn(1, small_config.d_model)
11    out2 = attn(x2, mask=mask)
12    # 前のつの位置は同じでなければなりません5
13    assert torch.allclose(out1[:, :5], out2[:, :5], atol=1e-5)

```

このテストに合格すると、因果マスクが正しく機能していることを確認できます。失敗した場合、マスクが

5.8 Self-Attention vs Cross-Attention

この章で実装したのは自己アテンション (self-attention) で、 Q, K, V がすべて同じシーケンスから来ます。cross-attention) も使用されます。これは Q がデコーダから、 K, V がエンコーダから来ます。GPTはデコーダのみ

5.8.1 クロスアテンションはどう違いますか？

クロスアテンションで $Q = x_{dec}W_Q$ 、 $K = x_{enc}W_K$ 、 $V = x_{enc}W_V$ 。デコーダの各位置がエンコーダの各位置とアテンションを計算する。GPTはこの交差アテンションを持っていません。自己アテンションだけで次のトークンを予測する。これがG

5.9 アテンション計算量分析

アテンションの計算量は $O(L^2 \cdot d)$ です。 L はシーケンス長です。シーケンスが長くなると、2乗に増加します。

シーケンス長	アテンション行列サイズ	FP16メモリ (シングルヘッド)
512	512×512	0.5 MB
2048	2048×2048	8 MB
8192	8192×8192	128 MB
32768	32768×32768	2 GB

これが長い文脈処理が難しい理由だ。第2巻第7章では、KVキャッシュとPagedAttentionでこのコストを削減

5.10 練習：アテンション重みの可視化

学習したモデルのアテンションの重みを視覚化すると、モデルがどのパターンに注目しているかがわかりま

```

1 import matplotlib.pyplot as plt
2 import math
3 from makellm.model.attention import make_causal_mask
4
5 # アテンション重みの手動計算と可視化 (は重みを内部的にのみ管理するため、デバッグ時に直接計算) SDPA
6 model.eval()
7 with torch.no_grad():
8     x = torch.randn(1, 10, model.config.d_model) # [batch=1, seq=10, d_model]
9     x_norm = model.blocks[-1].ln1(x)

```



```

10 qkv = model.blocks[-1].attn.qkv_proj(x_norm)
11 batch, seq, _ = x.shape
12 qkv = qkv.view(batch, seq, 3, model.config.n_heads, model.config.d_head).permute(2, 0, 3, 1, 4)
13 q, k = qkv[0], qkv[1] # [1, n_heads, seq, d_head]
14
15 # 番目のヘッドの重みの計算: (Q @ K^T) / sqrt(d_head)
16 scores = (q[0, 0] @ k[0, 0].T) / math.sqrt(model.config.d_head)
17 mask = make_causal_mask(seq)
18 attn_weights = torch.softmax(scores + mask, dim=-1) # [seq, seq]
19
20 plt.imshow(attn_weights.numpy(), cmap='viridis')
21 plt.colorbar()
22 plt.xlabel('key position')
23 plt.ylabel('query position')
24 plt.title('Attention weights (head 0, last layer)')

```

5.11 要約

- ・ アテンションは、クエリ - キー類似度で値を重み付けして合計する操作です。
- ・ $\sqrt{d_k}$ にスケールリングしてソフトマックスの飽和を防ぎます。
- ・ 因果マスクで将来トークンをブロックして自己回帰モデルになる。
- ・ マルチヘッドは d_{model} を h 個のヘッドに分割してさまざまな関係を学習。
- ・ QKVを一度に計算してGPU効率を最大化。
- ・ PyTorch 2.0のSDPAを活用すると、最適化されたアテンションを1行で使用可能。
- ・ アテンション計算量は $O(L^2)$ で長いシーケンスでボトルネック。

5.12 練習問題

1. $\sqrt{d_k}$ スケールリングを除くと、学習はどのように変わりますか？ $d_k = 64$ のときにsoftmax出力の分布を
2. 因果マスクをかぶらないとどうなりますか？学習時と推論時それぞれ説明しなさい。
3. ヘッド数 h を1にすると（シングルヘッド）マルチヘッドとどのような違いがあるのか？
4. ヘッド数 h を d_{model} に等しくすると（つまり $d_{head} = 1$ ）、何が起こりますか？
5. アテンション行列 $L \times L$ のメモリが $L = 100,000$ のときにいくらかを計算します（FP16ベース）。

次の章では、このアテンションを含むトランスブロックを組み立てます。アテンション + FFN + 残差 + L

Chapter 6

トランスフォーマーブロックの組み立て

この章では、5つのアテンションと4つの埋め込みを組み合わせてトランスフォーマーブロック (transformer block) を作成します。アテンションだけでは不足しており、FFN (Feed-Forward Network)、残差接続 (residual connection)、レイヤ正規化 (layer normalization) が一緒になって初めて意味のある表現を学習することが

6.1 変圧器ブロックの構造

1つのトランスブロックは、次の4つのコンポーネントで構成されています。

1. Multi-Head Attention: トークン間の情報交換
2. Feed-Forward Network (FFN): 位置別非線形変換
3. Residual Connection: 入力を出力に加えて深いネットワーク学習
4. Layer Normalization: 学習安定化

これらの4つがどのように組み合わせられるかがトランスブロックの核心です。1巻ではPre-LN構造 (GPT-2スタイル) を使用する。

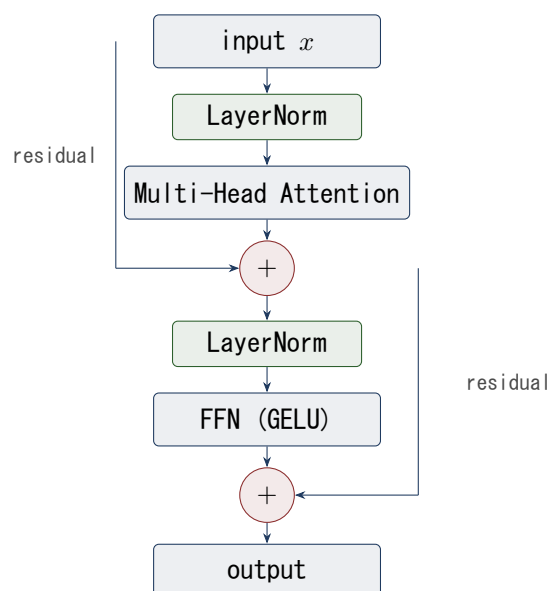


Figure 6.1: Pre-LN 変圧器ブロック構造. アテンションと FFN それぞれへの残差接続と LayerNormあり.

6.2 Feed-Forward Network (FFN)

アテンションはトークン間の「水平」情報交換を実行します。一方、FFNは各トークン位置で「垂直」非線形

$$\text{FFN}(x) = \text{Linear}_2(\text{GELU}(\text{Linear}_1(x)))$$

Linear_1 は $d_{\text{model}} \rightarrow d_{\text{ff}}$ に拡張し、 Linear_2 は $d_{\text{ff}} \rightarrow d_{\text{model}}$ に戻ります。 d_{ff} は通常 $4d_{\text{model}}$ であり「拡張-縮小」パターンは非線形表現力を与えます。

6.2.1 直観：なぜ拡張-縮小？

$d_{\text{model}} = 512$ を $d_{\text{ff}} = 2048$ に拡張して再び512に縮小するのは、「より大きな表現空間での操作」です。高次元空間での操作は、非線形表現力を与えます。実際、FFNは「キー値メモリ」として機能するという研究結果（Geva et al. 2021）もあります。 Linear_1 の

```
1 import torch.nn.functional as F
2
3 class FeedForward(nn.Module):
4     def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1):
5         super().__init__()
6         self.w1 = nn.Linear(d_model, d_ff)
7         self.w2 = nn.Linear(d_ff, d_model)
8         self.dropout = nn.Dropout(dropout)
9         self._init_weights()
10
11     def _init_weights(self) -> None:
12         nn.init.normal_(self.w1.weight, mean=0.0, std=0.02)
13         nn.init.normal_(self.w2.weight, mean=0.0, std=0.02)
14         if self.w1.bias is not None:
15             nn.init.zeros_(self.w1.bias)
16         if self.w2.bias is not None:
17             nn.init.zeros_(self.w2.bias)
18
19     def forward(self, x: torch.Tensor) -> torch.Tensor:
20         """x: [batch, seq, d_model] -> [batch, seq, d_model]"""
21         return self.dropout(self.w2(F.gelu(self.w1(x))))
```

6.3 GELU vs ReLU

元のトランスはReLUを使用していましたが、GPT-2以降はGELU(Gaussian Error Linear Unit)が標準です。GELUはReLUの「冷たい」0/1しきい値の代わりにガウス累積分布を使用してスムーズに切り替えます。

GELU

$$\text{GELU}(x) = x \cdot \Phi(x) \approx 0.5x \left(1 + \tanh \left[\sqrt{2/\pi} (x + 0.044715x^3) \right] \right)$$

$\Phi(x)$ は標準正規分布の累積分布関数です。 $x < 0$ でも小さい負の値を許容するので、勾配の流れが良いです。

6.3.1 なぜ GELUもっと良い？

ReLUは $x < 0$ で完全にゼロになり、グラデーションもゼロになります（「死んだReLU」の問題）。GELUは小さな負の値を受け入れ、グラデーションが常に流れます。また、切り替え点が滑らかで微分可能です。

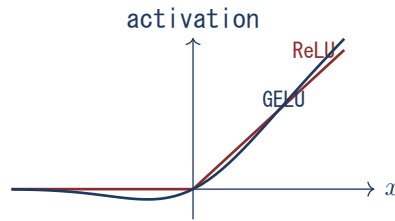


Figure 6.2: ReLU vs GELU. ReLUは $x \leq 0$ で正確にゼロですが、 $x > 0$ で $y = x$ と振る舞います。GELUは小さい負の値を許容してスムーズに切り替える。

実際、GPT-2、BERT、RoBERTaなど、ほとんどの現代モデルはGELUを使用しています。2巻では、SwiGLUとい

6.4 残差接続(Residual Connection)

残差接続 (residual connection) は入力を出力に直接加算する演算です。 $y = x + f(x)$ 。この単純なアイデアは He et al. (2016) のResNetが画像分類で100層以上の学習を可能にしたのがその始まりだ。

6.4.1 直観：なぜ残差が役に立つのか？

残差がない場合、ネットワークは $f(x) = y$ を学習する必要があります。深くなるほど、このマッピングを学習するのは難しくなります。 $y - x$ 、つまり「残差」だけを学習します。入力 x がすでに y に近い場合、 f は小さな補正だけです。

また、逆伝播時にグラデーションが「近道」を通して直接流れます。 $y = x + f(x)$ の微分は $\frac{dy}{dx} = 1 + f'(x)$ です。 $f'(x) = 0$ でもグラデーションは1になって消えません。

トランスフォーマブロックには2つの残差接続があります。アテンションの1つ、FFNの1つ。GPTは12~96階な

残差のない100階ネットワーク

実験的に残差を除いて100層ネットワークを学習すれば損失は減らない。グラデーションが逆伝播の途中消

6.5 Pre-LN vs Post-LN

LayerNormの位置に応じて2つのバリエーションがあります。

6.5.1 Post-LN (Vaswani オリジナル)

元のトランスはLayerNormをアテンション/FFN裏に配置しました：

$$x = \text{LN}(x + \text{Attn}(x))$$

しかし、この構造は深いネットワークで学習が不安定である。初期化に非常に敏感で、warmupがなければ発

6.5.2 Pre-LN (GPT-2スタイル)

LayerNormをアテンション/FFNフロントに配置します。

$$x = x + \text{Attn}(\text{LN}(x))$$

この構造は、残差パスにLayerNormがないため、グラデーションがよりよく流れます。深いネットワークでも

6.5.3 なぜ Pre-LNこれはより安定していますか？

Post-LNで残差パス $x \rightarrow x + \text{Attn}(x) \rightarrow \text{LN}(\cdot)$ を通過すると、グラデーションはLayerNormを通過する必要がある。LayerNormは分散に分割する操作であり、グラデーションをスケーリングします。

Pre-LNは、残差経路 $x \rightarrow x + \text{Attn}(\text{LN}(x))$ から x がLayerNormを経由せずに直接流れます。アテンション出力のみLayerNormを経た後に加算される。グラデーションがidentityパスを通じてそのまま流れ、深いネットワーク

```
1 class TransformerBlock(nn.Module):
2     def __init__(self, config: ModelConfig, layer_norm_style: str = "pre"):
3         super().__init__()
4         self.layer_norm_style = layer_norm_style
5         self.ln1 = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
6         self.ln2 = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
7         self.attn = MultiHeadAttention(config)
8         self.ffn = FeedForward(config.d_model, config.d_ff, config.dropout)
9
10    def forward(self, x, mask=None):
11        if self.layer_norm_style == "pre":
12            # Pre-LN: 正規化 -> アテンション -> 残差
13            x = x + self.attn(self.ln1(x), mask=mask)
14            x = x + self.ffn(self.ln2(x))
15        else:
16            # Post-LN: アテンション -> 残差 -> 正規化
17            x = self.ln1(x + self.attn(x, mask=mask))
18            x = self.ln2(x + self.ffn(x))
19        return x
```

Pre-LNを書く理由

Post-LNは、初期勾配が残差経路ではなくLayerNormを介してのみ流れているため、深いネットワークからPre-LNは残差経路に何もなく、グラデーションがidentityに流れ、100階以上でも学習が可能だ。ただし、LNは最後に追加のLayerNormが必要です（次の章でln_fで実装）。

6.6 残差スケーリング

GPT-2は、残差ストリームに直接寄与する重み（アテンションの W_O 、FFNの W_2 ）をより小さく初期化します。

```
1 def _init_residuals(self, module: nn.Module) -> None:
2     """残差ストリームに直接寄与する投影重みを小さく初期化."""
3     if isinstance(module, nn.Linear):
4         if module.weight.shape[0] == self.config.d_model:
5             # 残差に入る出力: より小さい標準偏差で初期化
6             nn.init.normal_(module.weight, mean=0.0,
7                             std=0.02 / math.sqrt(2 * self.config.n_layers))
```

$1/\sqrt{2N}$ スケーリングは、 N 層の場合の残差出力の分散が層数に比例して大きくなることを補正します。 N が

6.6.1 なぜ - 認可？

トランスブロックごとに残差に寄与する出力が2つ（アテンション、FFN）だ。 N 個のブロックの場合、合計 $1/\sqrt{2N}$ にする必要があります。

6.7 ブロックテスト

```
1 def test_block_shape(small_config):
2     block = TransformerBlock(small_config)
3     x = torch.randn(2, 8, small_config.d_model)
4     out = block(x)
5     assert out.shape == (2, 8, small_config.d_model)
6
7 def test_block_residual_connection(small_config):
8     """残差接続がある場合、入力と出力の差は有限でなければなりません."""
9     block = TransformerBlock(small_config)
10    block.eval()
11    x = torch.randn(2, 8, small_config.d_model)
12    out = block(x)
13    diff = (out - x).abs().mean()
14    assert diff.item() < 5.0 # 残差が大きすぎると正規化の問題
15
16 def test_feedforward_shape(small_config):
17     ffn = FeedForward(small_config.d_model, small_config.d_ff)
18     x = torch.randn(2, 8, small_config.d_model)
19     out = ffn(x)
20     assert out.shape == x.shape
```

6.8 変圧器ブロックの計算量

ブロック1つのFLOPsは約:

- ・ QKVプロジェクション: $3 \cdot L \cdot d^2$
- ・ アテンションスコア: $L^2 \cdot d$
- ・ アテンション出力投影: $L \cdot d^2$
- ・ FFN: $2 \cdot L \cdot d \cdot d_{ff} = 8Ld^2$ (since $d_{ff} = 4d$)

合計 $O(L \cdot d^2)$ であり、アテンションの $O(L^2 d)$ は $L < d$ の場合は無視できます。実際のGPT-3 ($d = 12288$, $L = 2048$) では、 $L \cdot d^2 = 3 \times 10^{11}$ 、 $L^2 \cdot d = 5 \times 10^{10}$ とFFNがより支配的です。

6.9 要約

- ・ トランスフォーマブロック = アテンション + FFN + 残差接続 + LayerNorm.
- ・ FFN は位置別の非線形変換 (Linear→GELU→Linear).
- ・ GELUはReLUよりもスムーズな移行で学習安定性を向上。
- ・ 残差接続は深いネットワーク学習の中核です。
- ・ Pre-LNがPost-LNより学習安定性が良く、現代LLM規格。
- ・ 残差スケール $1/\sqrt{2N}$ で深いネットワークの分散爆発を防ぎます。

6.10 練習問題

1. FFNの d_{ff} を $4d_{model}$ の代わりに d_{model} にすると、何が起こりますか？

2. 残差接続を削除して12層モデルを学習すると、どのような現象が観察されますか？
 3. Pre-LNで最後のブロックの後に追加のLayerNormが必要なのはなぜですか？
 4. GELUの代わりにSigmoid Linear Unit (SiLU、 $x \cdot \sigma(x)$) を使用すると、どのような違いがありますか？
 5. 残差スケールリングなしで100階のモデルを初期化すると、学習の初期に何が起こりますか？
- 次の章では、このブロックを N 層積み上げて完全な GPT モデルを作成し、学習ループを構築する。

Chapter 7

GPT モデルの完成と学習ループ

この章では、6つのトランスブロックを N 層積み上げて完全なGPT モデルを作成し、このモデルを学習するループを実装する。

7.1 GPT モデル構造

GPTはトランスデコーダを N 層積み重ねた自己回帰言語モデルです。全体の構造は次のとおりです。

1. トークンID→[Token + Position Embedding]

Transformer Block $\times N$

Final LayerNorm

LM Head: $d_{model} \rightarrow V$

2. logits

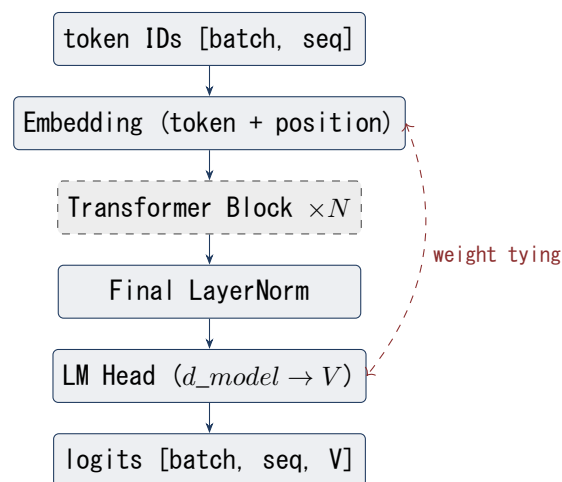


Figure 7.1: GPT モデル構造. 埋め込みの重みと LM Head 重みを共有する (weight tying) によるパラメータの節約.

7.2 ウェイトタイ (Weight Tying)

埋め込み行列 E ($V \times d_{model}$) とLM Head行列 W_{out} ($d_{model} \times V$) には事実のような情報が含まれています。2つを共有すると、パラメータの数を $V \times d_{model}$ だけ節約できます。

```
1 class GPT(nn.Module):
2     def __init__(self, config: ModelConfig):
3         super().__init__()
4         self.config = config
```



```

5     self.embedding = EmbeddingStack(
6         vocab_size=config.vocab_size,
7         d_model=config.d_model,
8         max_len=config.context_length,
9         pad_id=config.pad_token_id,
10    )
11    self.blocks = nn.ModuleList([
12        TransformerBlock(config, layer_norm_style="pre")
13        for _ in range(config.n_layers)
14    ])
15    self.ln_f = nn.LayerNorm(config.d_model, eps=config.layer_norm_eps)
16    self.lm_head = nn.Linear(config.d_model, config.vocab_size, bias=False)
17    # ウェイトタイ
18    self.lm_head.weight = self.embedding.token_emb.embedding.weight

```

重みタイの長所と短所

利点: パラメータの節約 (GPT-2小基準で約3分の1減少)、埋め込みと出力の一貫性。

欠点: 埋め込みが出力レイヤーのグラデーションも受け取り、学習がより複雑になります。一部のモデル (GPT-2、GPT-3) はタイ使用。LLaMA、Mistral は未使用。この本では、GPTスタイルに沿ってタイを使用しま

7.3 Forward Pass

```

1     def forward(self, token_ids, return_hidden=False):
2         batch, seq = token_ids.shape
3         # 因果マスクの作成
4         mask = make_causal_mask(seq, device=token_ids.device)
5         # 埋め込み
6         x = self.embedding(token_ids)
7         # トランスフォーマブロック通過
8         for block in self.blocks:
9             x = block(x, mask=mask)
10        # 最終正規化 (Pre-だから最後に必要) LN
11        x = self.ln_f(x)
12        # ロジット
13        logits = self.lm_head(x)
14        if return_hidden:
15            return logits, x
16        return logits

```

なぜ最後にLayerNormが必要なのですか？

Pre-LNブロックは各ブロックの入力を正規化します。しかし、最後のブロックの出力は、正規化されていない (LayerNorm) が必要です。

Post-LNでは、各ブロック出力はすでに正規化されているため、ln_fは不要です。

7.4 データセット: スライドウィンドウ

言語モデリングデータセットはテキストをトークン化した後、固定長チャンクに分割します。各チャンクで 1 トークン、target は最後の $L - 1$ トークン (一行シフト)。

```

1     class LMDataset(Dataset):

```

```

2  def __init__(self, text, tokenizer, context_length=128,
3      add_bos=True, add_eos=True):
4      self.context_length = context_length
5      # テキストをトークン化して長いシーケンスに
6      if isinstance(text, str):
7          text = [text]
8      all_ids = []
9      for t in text:
10         ids = tokenizer.encode(t, add_bos=add_bos, add_eos=add_eos)
11         all_ids.extend(ids)
12     self._ids = all_ids
13     # context_lengthサイズチャンクに分割+1
14     cl = context_length + 1
15     self._chunks = []
16     for i in range(0, len(all_ids), cl):
17         chunk = all_ids[i:i+cl]
18         if len(chunk) < cl:
19             chunk = chunk + [tokenizer.special.pad_id] * (cl - len(chunk))
20         self._chunks.append(chunk)
21
22     def __getitem__(self, idx):
23         chunk = self._chunks[idx]
24         input_ids = torch.tensor(chunk[:-1], dtype=torch.long)
25         target_ids = torch.tensor(chunk[1:], dtype=torch.long)
26         return input_ids, target_ids

```

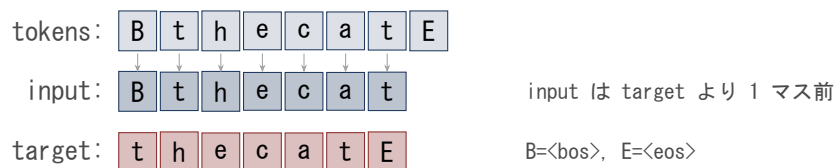


Figure 7.2: スライドウィンドウデータセット。 inputと target銀のようなシーケンス shiftー。モデルは inputを見て targetを予測。

7.5 損失関数: Cross-Entropy

言語モデルの損失はクロスエントロピー (cross-entropy) です。モデルが予測した確率分布と正解 (ワンホ

Cross-Entropy Loss

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(y_i | x_{<i})$$

y_i は正解トークン、 P はモデルの予測確率です。低いほど良いです。

```

1  def cross_entropy_loss(logits, targets, ignore_index=-100):
2      # logits: [batch, seq, V], targets: [batch, seq]
3      batch, seq, vocab = logits.shape
4      logits_flat = logits.view(-1, vocab)
5      targets_flat = targets.view(-1)
6      return F.cross_entropy(logits_flat, targets_flat, ignore_index=ignore_index)

```

7.5.1 Label Smoothing (選択)

正解に100%確信する代わりに、多少の不確実性を許容して一般化性能を高める技法。

```
1 def label_smoothing_loss(logits, targets, smoothing=0.1, ignore_index=-100):
2     """ 正解トークンに1-smoothing 確率, 残り smoothing/vocab 確率 """
3     log_probs = F.log_softmax(logits.view(-1, logits.size(-1)), dim=-1)
4     targets_flat = targets.view(-1)
5     nll_loss = -log_probs.gather(1, targets_flat.clamp(min=0).unsqueeze(1)).squeeze(1)
6     smooth_loss = -log_probs.mean(dim=-1)
7     mask = (targets_flat != ignore_index).float()
8     nll_loss = (nll_loss * mask).sum() / mask.sum().clamp(min=1)
9     smooth_loss = (smooth_loss * mask).sum() / mask.sum().clamp(min=1)
10    return (1.0 - smoothing) * nll_loss + smoothing * smooth_loss
```

7.6 オプティマイザー: AdamW

AdamWはAdamにweight decayを組み合わせた最適化アルゴリズムです。コアはバイアスとLayerNormにはweight decayを適用しないこと。

```
1 def build_optimizer(model, config):
2     # パラメータグループの分離: decay / no_decay
3     decay, no_decay = [], []
4     for name, p in model.named_parameters():
5         if not p.requires_grad: continue
6         if p.ndim < 2 or "bias" in name or "ln" in name.lower():
7             no_decay.append(p)
8         else:
9             decay.append(p)
10    return AdamW(
11        [{"params": decay, "weight_decay": config.weight_decay},
12         {"params": no_decay, "weight_decay": 0.0}],
13        lr=config.learning_rate, betas=(0.9, 0.95)
14    )
```

なぜバイアスとLayerNormはweight decayから減算するのですか？

Weight decayは重みをゼロに引き付ける正規化だ。行列の重みには過適合防止効果があるが、バイアスとLayerNormのweight decayの効果が微小であるのではなく学習を妨げる可能性がある。
PyTorchコミュニティの標準的な慣行です。

7.7 学習率スケジューラ

学習の初期には、学習率を徐々に上げ(warmup)、その後コサイン曲線に減少させる。このwarmup + cosineスケジューラはGPT-3で標準として使用されます。

```
1 def build_scheduler(optimizer, config, total_steps):
2     warmup = config.warmup_steps
3     def lr_lambda(step):
4         if step < warmup:
5             return float(step) / max(1, warmup)
6         progress = (step - warmup) / max(1, total_steps - warmup)
7         if config.lr_scheduler == "cosine":
```

```

8         return 0.5 * (1.0 + math.cos(math.pi * min(progress, 1.0)))
9     return 1.0
10    return LambdaLR(optimizer, lr_lambda)

```

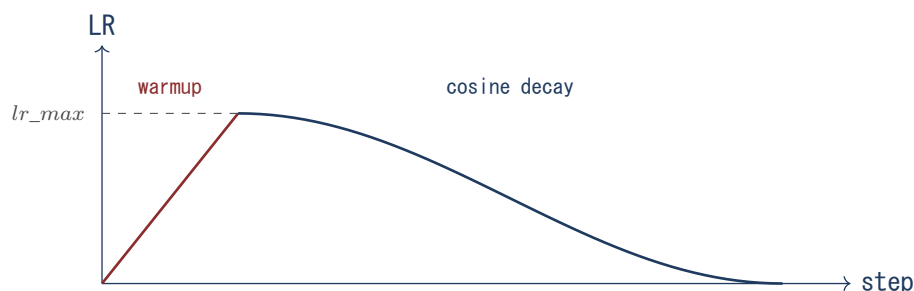


Figure 7.3: Warmup + Cosine 学習率スケジュール。最初は線形に上げます。その後、コサイン曲線に減少。

7.7.1 なぜ warmup が必要ですか？

学習初期には重みがランダムに近い。大きな学習率をすぐに適用すれば、グラデーションが不安定で発散する。特にアテンションの初期グラデーションは非常に大きくなることがあります（softmax飽和など）、warmupがPre-LN構造はPost-LNよりもウォームアップに敏感ではありませんが、それでも使用するのは安全です。

7.8 グラデーションクリッピング

グラデーションが大きすぎて重みが大きく変化するのを防ぐために、グラデーションnormを制限します。

```

1 if config.grad_clip > 0:
2     torch.nn.utils.clip_grad_norm_(model.parameters(), config.grad_clip)

```

通常`grad_clip = 1.0`を使用してください。グラデーションノームが1.0を超える場合は1.0にスケールリングします。

7.9 トレーナー

すべてのコンポーネントをまとめて学習ループを作成します。

```

1 class Trainer:
2     def __init__(self, model, train_dataset, config, model_config, eval_dataset=None):
3         self.model = model
4         self.config = config
5         self.device = torch.device(config.device)
6         self.model.to(self.device)
7         self.train_loader = DataLoader(train_dataset, batch_size=config.batch_size,
8                                       shuffle=True, collate_fn=collate_batch)
9         # 総ステップ数の計算
10        total_steps = len(train_dataset) // config.batch_size * config.max_epochs
11        self.optimizer = build_optimizer(model, config)
12        self.scheduler = build_scheduler(self.optimizer, config, total_steps)
13        # ...
14
15    def train(self):
16        self.model.train()
17        for inputs, targets in self.train_loader:

```

```

18         inputs, targets = inputs.to(self.device), targets.to(self.device)
19         logits = self.model(inputs)
20         loss = cross_entropy_loss(logits, targets,
21                                   ignore_index=self.model_config.pad_token_id)
22         self.optimizer.zero_grad(set_to_none=True)
23         loss.backward()
24         if self.config.grad_clip > 0:
25             torch.nn.utils.clip_grad_norm_(self.model.parameters(),
26                                             self.config.grad_clip)
27         self.optimizer.step()
28         self.scheduler.step()

```

7.10 Perplexity

パープレクサティ (Perplexity, PPL) は言語モデルの代表的な評価指標です。

Perplexity

$$\text{PPL} = \exp \left(\frac{1}{N} \sum_{-i} \log P(y_{-i} | x_{< i}) \right) = \exp(\text{loss})$$

PPLは、モデルが次のトークンを予測するときに「平均的にいくつかの候補に迷うか」を示します。PPL=1なら完璧、PPL=10なら10個候補に絞ることができるという意味。

PPLの解釈

- ・ PPL = 1: モデルは次のトークンを 100% 確信しています。理論的最小。
- ・ PPL = 10: 10 個のトークンのいずれかに絞り込みます。言語モデルとして使えるレベル。
- ・ PPL = 100: 100 個のトークンのいずれか。ほぼランダム。
- ・ PPL = V (語彙サイズ) : 完全にランダム。学習しない。

GPT-2 smallのWikiText-103 PPLは約17。GPT-3 175Bは約11。

7.11 チェックポイントを保存/ロード

学習中にモデルの重みを定期的に保存し、中断後に再開したり、推論に使用したりできます。

```

1 def save_checkpoint(self, filename):
2     path = self.out_dir / filename
3     torch.save({
4         "model_state": self.model.state_dict(),
5         "optimizer_state": self.optimizer.state_dict(),
6         "scheduler_state": self.scheduler.state_dict(),
7         "global_step": self.global_step,
8         "model_config": self.model_config.to_dict(),
9         "trainer_config": self.config.to_dict(),
10    }, path)
11
12 def load_checkpoint(self, path):
13     ckpt = torch.load(path, map_location=self.device)
14     self.model.load_state_dict(ckpt["model_state"])
15     self.optimizer.load_state_dict(ckpt["optimizer_state"])

```

```
16 self.scheduler.load_state_dict(ckpt["scheduler_state"])
17 self.global_step = ckpt["global_step"]
```

7.12 要約

- ・ GPT = Embedding + Transformer Block $\times N$ + LayerNorm + LM Head.
- ・ 重みタイロ埋め込みとLMヘッド重みを共有することでパラメータを節約します。
- ・ Pre-LN構造では、最後にln_fが必要です。
- ・ 言語モデリングは、スライドウィンドウで (input、target) ペアを生成します。
- ・ 損失はcross-entropy、オプティマイザはAdamW (bias/LayerNormはweight decayを除く)。
- ・ Warmup + Cosine 学習率スケジュール + グラデーショナルクリッピングが標準。
- ・ Perplexity =exp(loss)によるモデル品質評価。

7.13 練習問題

1. ウェイトタイを使用しないと、GPT-2 small (V = 50257、d = 768) でパラメータがどれだけ増加しますか？
2. Warmupステップ数を0にすると学習がどのように変わりますか？
3. Label smoothingを0.3に強く適用すると、どのような効果がありますか？
4. グラデーショナルクリッピングなしで学習するとどのような現象が発生する可能性がありますか？
5. PPLが1.0のモデルが実際に存在する可能性がありますか？なぜですか？

次の章では、学習したモデルでテキストを生成する方法について説明します。Greedy、Top-K、Top-Pのサンプリングすべてを実装します。

Chapter 8

テキスト生成 – サンプリング戦略とデコード

この章では、学習したGPTモデルでテキストを生成するプロセスを実装します。単純なgreedyデコードからTop-K、Top-P、Temperatureサンプリングまで、さまざまな戦略を扱います。

8.1 自己回帰生成の原理

言語モデルの生成は自己回帰 (autoregressive) 方式で行われます。すなわち、一度に1つのトークンを生成

1. プロンプト $[w_1, \dots, w_k]$ で始まります。
2. モデルに入れて、次のトークン分布 $P(w_{k+1} | w_1, \dots, w_k)$ を取得します。
3. 分布から w_{k+1} をサンプリング (または argmax)。
4. シーケンスに追加: $[w_1, \dots, w_k, w_{k+1}]$ 。
5. 終了条件 (<eos> または 最大長) まで2~4を繰り返します。

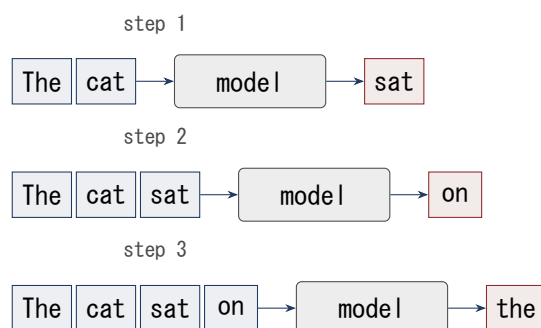


Figure 8.1: 自己回帰生成プロセス。ステップごとに前のトークンをモデルに入れ、次のトークンを予測。

8.2 Greedy サンプリング

最も単純な戦略は、各ステップで最も確率の高いトークンを選択することです。

```
1 class GreedySampler:
2     def __call__(self, logits: torch.Tensor) -> torch.Tensor:
3         # logits: [batch, vocab_size] -> token_ids: [batch]
4         return torch.argmax(logits, dim=-1)
```

Greedyは決定論的で高速ですが、毎回同じトークンだけを選択して単調なテキストが出る傾向があります。

「The cat sat on the mat. The cat sat on the mat. The cat...」同じ繰り返しに陥りやすい。

Greedyのトラップ：繰り返しループ

Greedyデコードは「安全な」トークンだけを選択して面白いテキストを作成します。より重大な問題は、「私は買い物を買うことができます。買い物をするができます...」同じ現象。

理由：モデルが「store」の後に「to buy」を高い確率で予測し、「to buy」の後に「some store」を高い確率で予測するループが形成されると、greedyはそこから抜け出せない。

8.3 Temperature サンプリング

分布の「鋭さ」を調整するパラメータを導入します。ロジットを温度 T で割った後のsoftmax。

Temperature Sampling

$$P_T(w) = \frac{\exp(\text{logit}(w)/T)}{\sum_{w'} \exp(\text{logit}(w')/T)}$$

- ・ $T < 1$: 分布が鋭くなる (greedyに近づく)
- ・ $T > 1$: 分布が平坦になる (よりランダム)
- ・ $T = 1$: 元の分布

```
1 class TemperatureSampler:
2     def __init__(self, temperature: float = 1.0):
3         assert temperature > 0
4         self.temperature = temperature
5
6     def __call__(self, logits):
7         probs = F.softmax(logits / self.temperature, dim=-1)
8         return torch.multinomial(probs, num_samples=1).squeeze(-1)
```

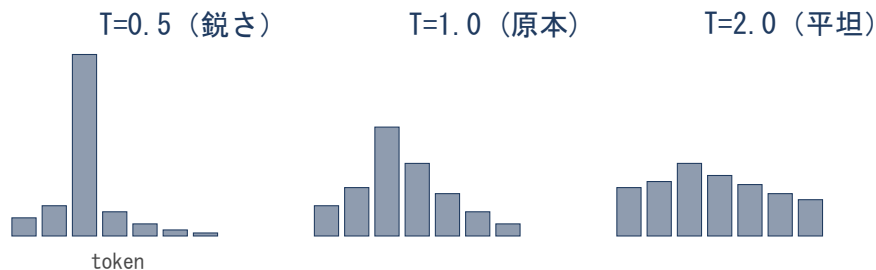


Figure 8.2: Temperatureによる分布の変化。 T が低いと分布が鋭くなります。 greedyに近づく, T 高いと平らになり、ランダム性が向上します。

8.4 Top-K サンプリング

確率の低いトークンが選択されるのを防ぐために、上位 K 個のトークンのみを考慮してください。

```
1 class TopKSampler:
2     def __init__(self, k: int = 50, temperature: float = 1.0):
3         self.k = k
4         self.temperature = temperature
5
6     def __call__(self, logits):
```



```

7     scaled = logits / self.temperature
8     top_values, top_indices = torch.topk(scaled, self.k, dim=-1)
9     probs = F.softmax(top_values, dim=-1)
10    sampled = torch.multinomial(probs, num_samples=1).squeeze(-1)
11    return top_indices.gather(-1, sampled.unsqueeze(-1)).squeeze(-1)

```

8.4.1 Top-Kの直観

ロジット分布から「尾」（確率が非常に低いトークン）を切り取ります。たとえば、語彙が5万個の場合、 $V=50000$ 、 $K=5$ の場合、 $K=1$ の場合はgreedyと同じです。Top-K = V （語彙サイズ）の場合は、temperatureサンプリングと同じ

8.5 Top-P（Nucleus）サンプリング

Top-Kは K を固定しますが、Top-Pは分布形式に従って考慮するトークンの数を動的に制御します。累積確率、

```

1 class TopPSampler:
2     def __init__(self, p: float = 0.9, temperature: float = 1.0):
3         self.p = p
4         self.temperature = temperature
5
6     def __call__(self, logits):
7         scaled = logits / self.temperature
8         probs = F.softmax(scaled, dim=-1)
9         sorted_probs, sorted_indices = torch.sort(probs, descending=True, dim=-1)
10        cumulative = torch.cumsum(sorted_probs, dim=-1)
11        # を超える位置からマスキングP
12        sorted_mask = cumulative - sorted_probs > self.p
13        sorted_probs = sorted_probs.masked_fill(sorted_mask, 0.0)
14        sorted_probs = sorted_probs / sorted_probs.sum(dim=-1, keepdim=True).clamp(min=1e-10)
15        sampled = torch.multinomial(sorted_probs, num_samples=1).squeeze(-1)
16        return sorted_indices.gather(-1, sampled.unsqueeze(-1)).squeeze(-1)

```

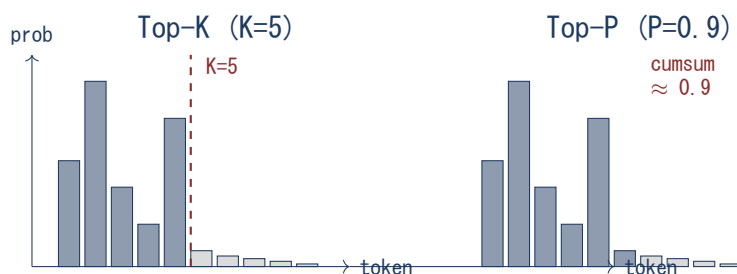


Figure 8.3: Top-K vs Top-P. Top-Kは固定数です，Top-P累積確率 P になるまで選択.

8.5.1 Top-K vs Top-P

考慮トークン数

分布がシャープである場合 & 多すぎるトークンを考慮し、少ないトークンのみを考慮する（自動）
分布が平らであるとき & あまりにも少ない考慮 & 多くのトークンを考慮する（自動）

適応性

Top-Pは通常より自然な結果を生み出します。モデルが確信する時は少ないトークンだけ、不確実な時は多く

8.6 サンプリング戦略の比較

戦略	多様性	一貫性	ケース
Greedy	Low	High	コード生成、数学
Temperature (T=0.7)	中	中	一般用途
Top-K (K=50)	中	高	チャットボット
Top-P (P=0.9)	中	高	チャットボット, 創作
Top-P (P=0.95)	高	中	創造的な執筆

8.7 生成関数の実装

すべてのサンプラーを組み合わせたgenerate関数を作成します。

```
1 @torch.no_grad()
2 def generate(model, tokenizer, prompt, max_new_tokens=50,
3             sampler=None, device="cpu", stop_on_eos=True):
4     if sampler is None:
5         sampler = GreedySampler()
6     model.eval()
7     model.to(device)
8     ids = tokenizer.encode(prompt, add_bos=True, add_eos=False)
9     input_ids = torch.tensor([ids], dtype=torch.long, device=device)
10    eos_id = tokenizer.special.eos_id
11    context_length = model.config.context_length
12
13    for _ in range(max_new_tokens):
14        # コンテキスト長制限
15        x = input_ids[:, -context_length:] if input_ids.shape[1] > context_length else input_ids
16        logits = model(x)
17        next_logits = logits[:, -1, :] # 最後の場所
18        next_id = sampler(next_logits)
19        input_ids = torch.cat([input_ids, next_id.unsqueeze(0)], dim=1)
20        if stop_on_eos and next_id.item() == eos_id:
21            break
```

```
22 return tokenizer.decode(input_ids[0].tolist())
```

8.8 コンテキスト長制限

モデルは学習時に決められた`context_length`までしか処理できません。生成中にシーケンスがこの長さを超える2巻では、この限界を克服するRoPEのextrapolation技術を扱う。

Sliding Windowの問題

コンテキストを切り取ると、モデルは「前部」を忘れます。 “ Once upon a time, there was a princess named Aurora. She lived in a castle. One day、[長い説明] ... Auro-
raは、「Aurora」が前の部分を切り取ると、「Aurora」がプリンセスの名前であるという文脈を失います
解決策：より長いコンテキストモデルの使用（RoPE extrapolation）、または要約による圧縮。 2巻で扱う。

8.9 サンプラーテスト

```
1 def test_greedy_picks_max():
2     sampler = GreedySampler()
3     logits = torch.tensor([[1.0, 3.0, 2.0, 0.5]])
4     out = sampler(logits)
5     assert out.item() == 1 # 最大値
6
7 def test_top_k_limits_choices():
8     torch.manual_seed(0)
9     logits = torch.tensor([[1.0, 5.0, 4.0, 3.0, 0.1]])
10    sampler = TopKSampler(k=2, temperature=1.0)
11    for _ in range(20):
12        idx = sampler(logits).item()
13        assert idx in (1, 2) # いつも topのいずれか-2
14
15 def test_top_p_limits_choices():
16     torch.manual_seed(0)
17     logits = torch.tensor([[0.0, 10.0, 9.0, 0.0, 0.0]])
18     sampler = TopPSampler(p=0.5, temperature=1.0)
19     for _ in range(20):
20         idx = sampler(logits).item()
21         assert idx == 1 # 最も確率の高いトークンのみ
```

8.10 練習：各種サンプリング比較

`scripts/tiny_train.py` デモを実行すると、5つのサンプリング戦略の出力を比較できます。

```
cd make-llm-basic
python scripts/tiny_train.py --epochs 5 --vocab 500
```

出力例：

```
1 プロンプト
2 : 'the'
```

```
3 [Greedy ] the cat sat on the mat
4 [Temperature=0.5 ] the lazy dog sleeps
5 [Temperature=1.0 ] the brown fox jumps
6 [Top-K=10 ] the quick fox runs
7 [Top-P=0.9 ] the warm sun shines
```

8.11 高度な復号技術(紹介)

この本では実装しませんが、実際のシステムで使用される高度な技術を紹介します。

8.11.1 Beam Search

各ステップで K の候補シーケンスを維持し、最後に最高のスコアのシーケンスを選択します。翻訳、要約な

8.11.2 Speculative Decoding

小さな「ドラフト」モデルが最初に K 個のトークンを生成すると、大きなモデルが並列に検証されます。当

8.11.3 Contrastive Search

繰り返しを避けながら一貫性を保つ最新の技法。以前のトークン表現との類似度でペナルティを与えて様々

8.12 要約

- ・ 自己回帰生成は一度に1つのトークンを生成し、シーケンスを拡張します。
- ・ Greedy : 最も確率の高いトークン。単調さ、繰り返しループの危険。
- ・ Temperature : 分布の鋭さを調整します。↑ 低いほど決定論的。
- ・ Top-K : 親 K 個だけを考慮。シンプルだが非適応。
- ・ Top-P : 累積確率 P までのトークンのみを考慮します。分布に適応。
- ・ コンテキスト長を超えると、最も古いトークンを切り捨てます。
- ・ Speculative decoding、beam searchなど高度な技術も存在。

8.13 練習問題

1. Greedyデコードが繰り返しループに入る理由を分析してください。どのような分布の場合、ループは?
2. Temperatureが0に近づくと、greedyとどう違うのですか? 0の場合、どのようなことが起こりますか?
3. Top-KとTop-Pを同時に適用するとどうなりますか? (すなわち、Top-K候補のうちTop-Pフィルタリング)
4. チャットボットにはどのサンプリングが最適ですか? 理由を説明してください。
5. Speculative decodingが速い理由は何ですか?

これで1冊のコア実装が終わった。次章では全体をまとめ、2巻で取り上げる上級テーマを紹介する。

Chapter 9

クリーンアップと次のステップ

おめでとうございます。ここまで従ったなら、読者は今トークナイザーからテキスト生成までLLMのすべてのLLM-advancedで取り上げる上級トピックを紹介します。

9.1 1巻まとめ

9.1.1 実装したもの

章	実装内容
第3章 & BPE、Unigram トークナイザー（学習/エンコード/デコード） 第4章 & Token / Positional Embedding、EmbeddingStack 第5章	Scaled Dot- Product Attention, Causal Mask, Multi- Head Attention
第6章 & FeedForward (GELU)、トランスフォーマーブロック (Pre-LN) 第7章 & GPTモデル、LMDataset、Cross-Entropy、AdamW、Cosine LR、Trainer 第8章 & Greedy / Temperature / Top-K / Top-P サンプラー、generate関数	

9.1.2 テスト

合計48のユニットテストがすべて合格します。形状テスト、数値テスト、統合テストにより、すべてのモジ

```
$ pytest tests/ -v
===== 48 passed in 2.47s =====
```

9.1.3 読者が持っているもの

これで読者は次のことができます。

- ・ 自分の言葉でトークナイザーを学ぶ
- ・ 任意のサイズGPTモデルを作成する (configのみを変更できます)
- ・ CPU/GPUで学習ループを実行する

- ・ さまざまなサンプリング戦略でテキストを生成する
- ・ HuggingFace Transformersのコードを読むときの内部動作を理解する

9.2 1巻の限界

正直に言えば、1巻で作ったモデルはChatGPTとは遠い。限界は大きく3つある。

サイズ：1巻モデルは1000万~1億パラメータレベルです。GPT-3は1,750億、GPT-4は推定1.7兆パラメータだ

アーキテクチャ：1巻は2017年のオリジナルトランスフォーマーに近い。現代LLM（LLaMA、Mistral、GPT-4）は、RoPE、GQA、SwiGLU、RMSNormなどのより効率的で信頼性の高いコンポーネントを使用しています。

ソート：第1巻モデルは「次のトークン予測」だけを学びました。ユーザーの質問に答えるチャットボットにはFine-Tuning）、RLHF（Reinforcement Learning from Human Feedback）、DPO（Direct Preference Optimization）などのソート技術が必要です。

9.3 2巻で扱うこと

2巻Make LLM-advancedは、上記3つの制限を克服する方法を扱います。

9.3.1 分散学習 (2 - 第4章)

- ・ DDP (Distributed Data Parallel)：複数のGPUで同じモデルのレプリケーションを学習します。
- ・ FSDP (Fully Sharded Data Parallel)：モデルをシャードに分割することでメモリを節約します。
- ・ ZeRO-1/2/3：オプティマイザ/グラデーション/パラメータをシャーディング。
- ・ Pipeline Parallel：モデルを階層別に別のGPUに配置。
- ・ Tensor Parallel：単一レイヤーを複数のGPUに分割 (Megatron-LM)。

9.3.2 最新アーキテクチャ (5 - 第6章)

- ・ RoPE (Rotary Position Embedding)：複素数回転で位置エンコード。 extrapolation 優秀。
- ・ GQA (Grouped Query Attention)：KVヘッドを減らして推論メモリを節約します。
- ・ SwiGLU：GLU バリエーションによる FFN パフォーマンスの向上。 LLaMAが使用。
- ・ RMSNorm：LayerNorm の軽量化バージョン。
- ・ MoE (Mixture-of-Experts)：エキスパートネットワークプールでトークンごとの親K個のみを有効にし

9.3.3 量子化と推論の最適化 (7章)

- ・ INT8/INT4 量子化：重みを低ビット整数に変換することでメモリを4~8倍に節約。
- ・ AWQ：アクティブ値統計を利用した精度損失の最小化。
- ・ GPTQ：二次情報を使用した量子化。
- ・ KV Cache：以前のトークンのK、Vを再利用して推論を加速します。
- ・ PagedAttention：vLLM のページ単位の KV 管理によるメモリ断片化の解決。

9.3.4 ソートとファインチューニング（8枚）

- ・ SFT (Supervised Fine-Tuning) : 指導学習でチャットボット行動学習。
- ・ RLHF : 補償モデル+ PPOによる人間の好みの最適化。
- ・ DP0: RLHFの複雑さなしに直接の好みを最適化。
- ・ LoRA : 低ランクアダプタで効率的なファインチューニング。

9.3.5 データパイプライン（9章）

- ・ 品質フィルタ: 長さ、言語、繰り返し率でフィルタリングします。
- ・ MinHash 重複排除: Jaccard類似度近似による重複文書の検出。
- ・ 合成データ: テンプレートベースの学習データの生成。

9.4 推奨学習パス

2冊に進む前におすすめのアクティビティ:

1. 小さなプロジェクト: 1冊のコードで自分の関心のあるテキスト（例えば、ウィキバックスと韓国語、Gi
10万トークン程度なら1時間以内に学習される。
2. 論文を読む: 「Attention Is All You Need」 (Vaswani et al. 2017)、「Language
Models are Few-Shot Learners」 (Brown et al. 2020) を読んでください。1冊を読んだので、はるか
3. HuggingFace コードを読む: transformers/models/gpt2/modeling_gpt2.pyを開きます。私たちが作ったコードに

9.5 最後に

LLMは複雑に見えますが、最終的にはトークナイザー、埋め込み、アテンション、FFN、学習ループの組み合わせ
2巻では、この基盤の上にChatGPTレベルの技術を積み上げるだろう。

コーディングを停止しないでください。実験を止めないでください。それが実力を育てる唯一の道だ。

1巻終わり

dirmich

Appendix A

数学基礎補足

この付録では、本文で使用した数学的概念を補足します。微積分と基礎線形代数を知るが行列微分などに慣

A.1 ベクトルと行列

ベクトルは数字の1次元配列です。 $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ と表記。行列は 2 次元配列。 $A \in \mathbb{R}^{m \times n}$ は m 行 n 列。

- ・ 内積 (dot product) : $\mathbf{a} \cdot \mathbf{b} = \sum_i a_i b_i$ 。スカラー戻り。
- ・ 行列積 : $C = AB$ where $C_{ij} = \sum_k A_{ik} B_{kj}$ 。
- ・ 転置 (transpose) : A^T は行と列を変更します。
- ・ norm : $\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}$ 。ベクトルの長さ。

A.2 Softmax

Softmaxは、任意の実数ベクトルを確率分布に変換する関数です。

Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

出力のすべての要素は0と1の間であり、合計は1です。アテンション重み、分類モデルの出力などに使用。

A.3 Cross-Entropy

クロスエントロピー (cross-entropy) は2つの確率分布 p, q の差を測定します。

Cross-Entropy

$$H(p, q) = - \sum_i p_i \log q_i$$

分類の問題では、 p は正解の1つのホットベクトル、 q はモデル予測softmax出力です。単一の正解 y に対して $-\log q_y$ に単純化。

A.4 連鎖法則と Backpropagation

連鎖法則 (chain rule) は合成関数の微分法です。

$$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$$

ニューラルネットワークは合成関数の連続であるため、連鎖法則を使用して入力から出力までの微分を計算
PyTorchはこのプロセスを自動化するので、私たちが直接微分式を導く必要はありません。しかし、「どの

A.5 行列微分

損失 $\mathcal{L}$ が行列 W に依存する場合、 $\partial\mathcal{L}/\partial W$ は W と同じ形状の行列であり、各要素は $\partial\mathcal{L}/\partial W_{ij}$ です。

Jacobianはベクトル関数の微分を表す行列です。 $y = f(x)$ のとき $J_{ij} = \partial y_i / \partial x_j$ 。

詳細については、「The Matrix Cookbook」(Petersen & Pedersen)を参照してください。

A.6 正規分布

正規分布 (normal distribution) $\mathcal{N}(\mu, \sigma^2)$ の確率密度関数:

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$$

重みの初期化によく使用されます。 $\mu = 0, \sigma = 1/\sqrt{d}$ を書くと、入力次元が大きくなっても出力分散が一定に

A.7 情報理論の基礎

エントロピー (entropy)は確率分布の「ランダム性」を測定します。

$$H(p) = -\sum_i p_i \log p_i$$

KL 発散 (KL divergence)は2つの分布の違いです。

$$D_{KL}(p||q) = \sum_i p_i \log \frac{p_i}{q_i}$$

Cross-entropyは $H(p)+D_{KL}(p||q)$ に分解されます。 $H(p)$ が固定なので、クロスエントロピーを最小化する

A.8 最適化の基礎

傾斜下降法 (gradient descent): 損失を減らす方向にパラメータを更新します。

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$$

η は学習率です。Adamは、運動量と適応学習率を組み合わせた最適化アルゴリズムです。AdamWはAdamにweight decayを組み合わせたバージョンで、LLM学習の標準です。

詳細な誘導は、Goodfellow et al. 「Deep Learning」4~第8章を参照してください。

Appendix B

コードの完全なリスト

この付録は、1巻で実装したコアモジュールの完全なコードをまとめたものです。本文では説明のために部分

B.1 プロジェクト構造

```
make-llm-basic/  
src/make-llm/  
  __init__.py  
tokenizer/  
  __init__.py  
  base.py      # Tokenizer 抽象クラス  
  bpe.py       # BPE トークナイザー  
  unigram.py   # Unigram トークナイザー  
model/  
  __init__.py  
  embedding.py # TokenEmbedding, PositionalEmbedding  
  attention.py # MultiHeadAttention, CausalMask  
  transformer_block.py  
  gpt.py       # GPT モデル  
training/  
  __init__.py  
  dataset.py   # LMDataset  
  loss.py      # cross_entropy_loss  
  optimizers.py # AdamW + cosine scheduler  
  trainer.py   # Trainer クラス  
inference/  
  __init__.py  
  sampler.py   # Greedy, Top-K, Top-P, Temperature  
  generate.py  # generate 機能  
utils/  
  __init__.py  
  config.py    # ModelConfig, TrainerConfig  
  seed.py      # set_seed  
  logging.py   # Logger  
tests/  
  # pytest ユニットテスト  
book/  
  # この本の LaTeX ソース
```

B.2 テストの実行

```
# パッケージのインストール  
cd make-llm-basic  
pip install -e .
```

```
# フルテストの実行
pytest tests/ -v

# 特定のモジュールのみテスト
pytest tests/test_tokenizer.py -v
pytest tests/test_model.py -v
pytest tests/test_training.py -v
```

B.3 クイックスタートの例

```
1 """冊のすべてのモジュールを使用した1 end-to-end 例."""
2 from makellm.tokenizer import BPETokenizer
3 from makellm.model import GPT
4 from makellm.training import LMDataset, Trainer
5 from makellm.inference import generate, TopPSampler
6 from makellm.utils import ModelConfig, TrainerConfig, set_seed
7
8 set_seed(42)
9
10 # 1. トークナイザー学習
11 corpus = ["the quick brown fox jumps", "the lazy dog sleeps"] * 100
12 tokenizer = BPETokenizer()
13 tokenizer.train(corpus, vocab_size=500)
14
15 # 2. モデルの作成
16 model_config = ModelConfig(
17     vocab_size=tokenizer.vocab_size,
18     context_length=32, d_model=64, n_heads=4, n_layers=2, d_ff=128
19 )
20 model = GPT(model_config)
21 print(f"Model: {model}")
22 print(f"Parameters: {model.num_parameters():,}")
23
24 # 3. データセットの準備
25 dataset = LMDataset(corpus, tokenizer, context_length=32)
26
27 # 4. 学習
28 trainer_config = TrainerConfig(
29     batch_size=4, learning_rate=3e-4, max_steps=100,
30     warmup_steps=10, device="cpu"
31 )
32 trainer = Trainer(model, dataset, trainer_config, model_config)
33 trainer.train()
34
35 # 5. テキスト生成
36 sampler = TopPSampler(p=0.9, temperature=0.8)
37 text = generate(model, tokenizer, prompt="the", max_new_tokens=20, sampler=sampler)
38 print(f"Generated: {text}")
```

B.4 モジュール別 API 要約

モジュール	主な API
makellm.tokenizer	BPETokenizer, UnigramTokenizer
makellm.model	GPT, MultiHeadAttention, TransformerBlock
makellm.training	LMDataset, Trainer, cross_entropy_loss
makellm.inference	generate, GreedySampler, TopKSampler, TopPSampler
makellm.utils	ModelConfig, TrainerConfig, set_seed

B.5 設定データクラス

```
1 @dataclass
2 class ModelConfig:
3     vocab_size: int = 1000
4     context_length: int = 128
5     d_model: int = 128
6     n_heads: int = 4
7     n_layers: int = 4
8     d_ff: int = 512
9     dropout: float = 0.1
10    use_bias: bool = True
11    layer_norm_eps: float = 1e-5
12    pad_token_id: int = 0
13
14 @dataclass
15 class TrainerConfig:
16     batch_size: int = 32
17     learning_rate: float = 3e-4
18     weight_decay: float = 0.1
19     max_epochs: int = 10
20     warmup_steps: int = 100
21     lr_scheduler: str = "cosine"
22     grad_clip: float = 1.0
23     log_every: int = 10
24     eval_every: int = 200
25     save_every: int = 1000
26     seed: int = 42
27     device: str = "cpu"
28     out_dir: str = "./runs"
```

これらの設定を調整して、さまざまなサイズのモデルを試すことができます。たとえば、GPT-2 small に似た設定は `d_model=768`, `n_heads=12`, `n_layers=12`, `context_length=1024` です（ただし、実際の学習にはかなり

B.6 ライセンス

コードはMITライセンスで配布されています。自由にコピー、修正、商業使用が可能です。本の本文はCC BY-NC-SA 4.0（著者確認が必要）。

Appendix C

よくある質問と罣

この付録は1冊を学習し、読者がよく経験する問題と質問をまとめたものである。各項目は、実際の実装中に

C.1 環境関連

C.1.1 Q: PyTorch インストール後 CUDAが認識されません。

A: 2つのことを確認してください。まず、NVIDIAドライバがインストールされているか（nvidia-smi実行）。
12.1用のPyTorchをCUDA 11.8システムにインストールすると認識されません。

```
# CUDA バージョン確認
nvidia-smi | grep "CUDA Version"

# PyTorch 再インストール (CUDA 用) 12.1
pip uninstall torch
pip install torch --index-url https://download.pytorch.org/whl/cu121
```

C.1.2 Q: 学習が遅すぎる

A: 以下を確認してください:

1. GPUを使用しているか（device="cuda"設定）。
2. バッチサイズが小さすぎないか（GPU使用率も低い）。
3. データのロードがボトルネックか（num_workers増加）。
4. 混合精度（BF16）を使用しているか。

C.2 トークナイザー関連

C.2.1 Q: BPEで学んだトークナイザー “こんにちは” 奇妙なトークン化

A: 韓国語がコーパスに十分に含まれていないため。BPEは頻度ベースであり、まれな文字はバイトに分解さ

- ・ 韓国語コーパスの割合を高めること。
- ・ 語彙サイズを増やすこと（韓国語音節約2万個＋サブワード）。
- ・ SentencePieceのようなプロのトークナイザーを使用。

C.2.2 Q: encode/decode ラウンドトリップが失敗しました。

A: 空白の処理を確認してください。BPEの「」文字がデコード時に空白に変換されない可能性があります。

```

1 def _postprocess(self, text):
2     text = text.replace("_", " ")
3     return text.strip()

```

C.2.3 Q: 特殊トークン IDが変わるとどうなりますか？

A: 致命的。モデルは<pad>=0, <bos>=1, <eos>=2, <unk>=3を想定して学習されました。トークナイザーを再学習しな

C.3 モデル関連

C.3.1 Q: アテンション出力 NaNこれが出ます。

A: 3つの原因が一般的です:

1. ソフトマックス飽和: ロジットが大きすぎてsoftmaxが0/1に飽和しました。スケーリング ($\sqrt{d_k}$) 確認
2. 学習率が大きすぎる: グラデーション爆発。学習率を1/10に減らしてください。
3. FP16オーバーフロー: アクティブ値がFP16の範囲 (± 65504) を超えています。BF16に切り替えます。

```

1 # NaN 確認
2 if torch.isnan(loss):
3     print("NaN detected! Check learning rate and gradients.")
4     print(f"Max grad: {max(p.grad.abs().max() for p in model.parameters() if p.grad is not None)}")

```

C.3.2 Q: モデルは同じ単語を繰り返します。

A: サンプリング戦略を確認してください。Greedyは繰り返しループに陥りやすいです。Top-P ($P = 0.9$) に切り替えるか、Temperatureを0.7~0.9に設定します。繰り返しペナルティの追加も効果的。

```

1 # 繰り返しペナルティ簡素化()
2 def apply_repetition_penalty(logits, generated_ids, penalty=1.2):
3     for token_id in set(generated_ids.tolist()):
4         logits[token_id] /= penalty
5     return logits

```

C.3.3 Q: コンテキスト長を超えるとエラーが発生します

A: モデルのcontext_lengthを超える入力を入れると埋め込みフェーズでエラー。generate関数からスライドウィ

```

1 if input_ids.shape[1] > context_length:
2     x = input_ids[:, -context_length:] # 最新のものを維持

```

C.4 学習関連

C.4.1 Q: 損失は減りません。

A: 以下を確認してください:

1. 学習率が小さすぎる ($1e-6$ 以下)、大きすぎる ($1e-2$ 以上)。
2. データが少なすぎます (最小数千のトークンが必要です)。
3. オプティマイザが無効です (AdamWのベータ、epsを確認)。

4. グラデーションが流れない (requires_grad チェック)。

```
1 # グラデーションフローの確認
2 for name, p in model.named_parameters():
3     if p.requires_grad and p.grad is None:
4         print(f"No gradient for {name}")
5     elif p.grad is not None and p.grad.abs().max() < 1e-10:
6         print(f"Vanishing gradient for {name}")
```

C. 4. 2 Q: 学習の初期に損失が発散します。

A: Warmupが不足しているか、初期化が間違っている可能性があります。以下を試してください:

- ・ Warmup ステップを増やす (100~1000)。
- ・ グラデーションクリッピングチェック (grad_clip=1.0)。
- ・ 初期化標準偏差を減らす (0.02→0.01)。

C. 4. 3 Q: チェックポイントをロードすると他の結果が表示されます。

A: 2つの原因:

1. シード未設定: set_seed(42) を呼び出したことを確認します。
2. ドロップアウト: モデルをeval()モードに切り替えたことを確認します。
3. CuDNN 非決定論: torch.backends.cudnn.deterministic = True.

C. 5 分散学習関連 (2巻プレビュー)

C. 5. 1 Q: DDPで学習すると速度がむしろ遅くなります。

A: 通信ボトルネックである可能性。以下を確認してください。

- ・ バッチサイズが小さすぎ、演算/通信比が低い。
- ・ NVLinkではなく、低速通信 (PCIeなど) を使用。
- ・ グラデーション同期がステップごとに起こります (gradient accumulation に減少)。

C. 5. 2 Q: FSDPから OOMこれは私です

A: アクティブメモリが不足。有効チェックポイントをオンにするか、バッチサイズを減らします。CPUオフロードも考慮。

C. 6 デバッグのヒント

C. 6. 1 段階的に検証する

複雑なバグは小さな単位で割って検証。トークナイザーのみ単独テスト、アテンションのみ単独テストなど

```
1 # トルクナイザー単独テスト
2 tok = BPETokenizer()
3 tok.train(["hello world"], vocab_size=50)
4 print(tok.encode("hello"))
5 print(tok.decode(tok.encode("hello")))
6
```

```

7 # アテンション単独テスト
8 attn = MultiHeadAttention(config)
9 x = torch.randn(1, 4, 32)
10 out = attn(x)
11 print(out.shape, out.dtype, torch.isnan(out).any())

```

C. 6. 2 shapeを常に出力する

バグの 80% は shape 不一致。あらゆる段階で形状を出力する習慣。

```

1 print(f"input shape: {x.shape}")
2 print(f"after embedding: {emb.shape}")
3 print(f"after attention: {attn_out.shape}")

```

C. 6. 3 小さな入力で始める

バッチ1、シーケンス4で始まり、動作を確認してサイズを大きくする。大きな入力でデバッグすると出力が

C. 7 パフォーマンス最適化のヒント

C. 7. 1 混合精度の使用

```

1 from torch.cuda.amp import autocast, GradScaler
2
3 scaler = GradScaler()
4 with autocast(dtype=torch.bfloat16):
5     logits = model(inputs)
6     loss = criterion(logits, targets)
7 scaler.scale(loss).backward()
8 scaler.step(optimizer)
9 scaler.update()

```

BF16はFP16とは異なり、スケーラを必要としません。 A100以上で推奨。

C. 7. 2 torch.compile

PyTorch 2. 0のtorch.compileでモデルをコンパイルすると、1. 5~の2倍の速度が向上します。

```

1 model = GPT(config)
2 model = torch.compile(model) # コンパイル

```

C. 7. 3 プロファイリング

```

1 with torch.profiler.profile(activities=[torch.profiler.ProfilerActivity.CPU,
2                                     torch.profiler.ProfilerActivity.CUDA]) as prof:
3     # 学習コード
4 prof.export_chrome_trace("trace.json")

```

Chromeブラウザでchrome://tracingで開き、ボトルネック分析。

Appendix D

練習問題の答え

この付録は本文練習問題の答えを提供します。自分で解放してから確認してほしい。

D.1 第1章 答え

1-1. ChatGPTの回答を評価してください。

ベストプラクティス：「言語モデルは次のトークン予測で学習されますが、十分に大きなモデルとさまざまな回答パターンを暗黙的に学習します。ユーザーが質問の形で入力すると、モデルは学習データから質問の回答を生成します。」

1-2. 「太陽は東から…」次の単語の確率。

人ならば、「湧き出る」を99%以上にする。理由：（1）日常的な観察、（2）言語的集団（「東から湧き出る」）

1-3. Nグラム of 希少性問題。

「ホッキョクグマが踊る」次の単語をNグラムが予測するには、学習データに「ホッキョクグマが踊る」次の単語が必要です。

1-4. トランスがRNNよりも並列処理に有利である理由。

RNNは、時点 t の計算が時点 $t-1$ の隠れ状態に依存するため、順次処理必須。トランスはアテンションですべてのトークンにアクセスできます。

1-5. Chinchilla 基準データスケール。

パラメータを10倍増やすと、データも約10倍増やす必要があります（20トークン/パラメータ比を維持）。つまり、データも10倍増やす必要があります。

D.2 第3章 答え

3-1. 頻度の低いペアをマージすると？

頻繁に登場しないパターンがトークンになり、語彙が非効率的。まれな「xz」のようなペアがマージされる。

3-2. Unigramペナルティ-5.0に変更。

語彙にない部分文字列のペナルティが小さくなるので、Viterbiがより積極的に分割を試みます。その結果、分割が増えます。

3-3. 韓国語BPE、語彙100対1000。

語彙100：「こんにちは」が5つ以上のバイトトークンに分解されました。非常に非効率的。語彙1000：「こんにちは」が1つのバイトトークンに分解されました。

3-4. Viterbiの長さの上限4に縮小。

5文字以上のトークンが消えた。「hello」のような単語が「he」、「ll」、「o」に分解され、シーケンスの長さが短くなります。

3-5. <bos>の必要性。

そうでなくてもモデルは動作しますが、シーケンスの開始を明示的に表示すると、モデルが「ここから新しいシーケンスを開始します」と認識します。

D.3 第4章 答え

4-1. 埋め込み次元が小さすぎるか大きい場合。

小さい場合 (8) : 単語間の区別が難しく、意味的な類似性を学習できません。大きな面 (4096) : パラメ

4-2. Sinusoidal 定数 100→100.

周期が短くなり、近い位置も区別しにくくなる。位置エンコーディングの「解像度」が低くなります。

4-3. スケーリングを省略。

位置埋め込みがトークン埋め込みを圧倒し、モデルが位置のみに依存。意味情報の損失。

4-4. 学習可能な位置埋め込み、max_len 超過。

インデックス範囲外でエラーが発生しました。RoPEのようなextrapolationが可能な方法が必要。

4-5. LayerNorm ϵ 値の変更。

ϵ が大きいと正規化が弱くなり、出力分散が大きくなります。小さすぎると分母がゼロに近く、数値不安定。

D.4 第5章 答え

5-1. スケーリングを省略した場合のsoftmax。

$d_k = 64$ のとき、内積値が ± 64 まで行くことができ、softmaxがワンホットに飽和。グラデーション0、学習停

5-2. 因果マスク未使用。

学習時：モデルは将来のトークンを「見て」正解になるので、学習損失は低くなりますが一般化しません。

推論時：将来のトークンがないため、学習 - 推論ギャップが発生しました。パフォーマンスの急落。

5-3. ヘッド数 1 (シングルヘッド)。

1つのアテンションパターンのみを学習。文法、意味、談話関係をしたヘッドがすべて処理しなければならな

5-4. ヘッド数 $= d_{model}$ ($d_{head} = 1$)。

各ヘッドが1次元のみ処理。アテンションの意味が消え、単純な加重になる。事実上線形変換と同じ。

5-5. アテンション行列メモリ、 $L = 100,000$ 。

$100000^2 \times 2\text{bytes (FP16)} = 20\text{ GB}$. シングルヘッド基準。ヘッドが32個あれば640GB。事実上不可能。

D.5 第6章 答え

6-1. $d_{ff} = d_{model}$.

FFNの拡大縮小パターンが消え、非線形表現力が減少しました。より深い線形変換に似ています。

6-2. 残差なしで12階。

backward 時グラデーションが前の階に到達できず学習不可。損失が減らない。

6-3. Pre-LNでln_fが必要です。

Pre-LNは各ブロック入力を正規化しますが、最後のブロック出力は正規化されていません。LMヘッドに入れ

6-4. GELUの代わりにSiLU。

性能差ミミ。SiLU ($x \cdot \sigma(x)$) もスムーズな切り替え。一部モデル (LLaMAのFFNはSwiGLUですが活性化として)
6-5. 残差スケールリングなしで100階。

初期活性値分散が層ごとに累積されて爆発。学習の初期にNaNが発生する可能性が高い。

D.6 第7章 答え

7-1. ウェイトタイ未使用、GPT-2小。

$V \times d = 50257 \times 768 \approx 38.6M$ パラメータを追加。合計124M→163M。

7-2. Warmup 0.

学習初期グラデーションが不安定で発散の可能性。特にPost-LNでは深刻。Pre-LNはそれほど敏感ではあり

7-3. Label smoothing 0.3.

強いスムージングで、モデルは正解に対する確信が少ない。一般化は向上しますが、学習損失が高く維持。

7-4. グラデーションクリッピングなし。

グラデーション爆発時の重みが大きく変化し、モデルが崩壊。特に大きな学習率で危険。

7-5. PPL 1.0 可能ですか？

理論的に不可能。自然言語は本質的に不確実性があり、次のトークンが100%予測不可。学習データに完全に

D.7 第8章 答え

8-1. Greedy 繰り返しループの原因。

分布が「store」の後に「to buy」を高く、「to buy」の後に「some store」を高く予測するループが形成さ

8-2. Temperature → 0.

$T = 0$ なら softmax が argmax になり greedy と同じ。 $T = 0.01$ の場合、ほぼgreedy。分布が鋭すぎて多様性
0.

8-3. Top-K後Top-P。

Top-Kで候補をK個に減らした後、その中から累積確率Pまで選択。より強力なフィルタリング。品質は向上し

8-4. チャットボットに適したサンプリング。

Top-P ($P=0.9$ 、 $T=0.7 \sim 0.9$)。多様性と一貫性のバランス。Greedyは単調で、純粋なサンプリングは一貫性不

8-5. Speculative decodingが速い理由。

小さなモデルがK個のトークンをすばやく生成すると、大きなモデルがそのK個を一度に検証（並列処理）。

Appendix E

高度なトピックのプレビュー

この付録では、2巻Make LLM-advancedで取り上げる高度なトピックを簡単に紹介します。1冊を終えた読者が

E.1 分散学習

1巻モデルは単一GPU/CPUで学習した。しかし、70B以上のモデルは単一のGPUメモリに入らない。

DDP (Distributed Data Parallel) : モデルのレプリカを各GPUに置き、データのみを分割します。最も単純で

FSDP (Fully Sharded Data Parallel) : パラメータ、グラデーション、オプティマイザの状態をすべてシャー
70B+モデル学習可能。

ZeRO : DeepSpeedの段階的なシャーディング。 ZeRO-1/2/3で徐々にメモリを節約。

パイプライン並列 : モデルを階層別にGPUに分割。ノード間通信に適しています。

テンソル並列 : 単一レイヤーの重みを分割します。 Megatron-LM方式

E.2 最新のアーキテクチャ

1巻は2017年のオリジナルトランスフォーマーに近い。最新のLLMはより効率的なコンポーネントを使用して

RoPE (Rotary Position Embedding) : 複素数回転で位置エンコード。 extrapolationは優れており、長いコン
LLaMA、Mistralを使用。

GQA (Grouped Query Attention) : KVヘッドを減らして推論メモリを節約します。 LLaMA-2
70BがGQA-8を使用。

SwiGLU: GLU バリエーションの活性化による FFN パフォーマンスの向上。

RMSNorm: LayerNorm の軽量バージョン。計算量 10~20% 減少。

MoE (Mixture-of-Experts) : 複数の専門家のうち、トークンごとに上位K個のみを有効にします。

Mixtral 8x7Bが47Bの全パラメータで13Bアクティブに動作。

E.3 量子化と推論の最適化

INT8/INT4 量子化: 重みを低ビット整数に変換することでメモリを4~8倍節約。推論コスト決定的削減

AWQ: アクティブ値統計による重要なチャネルの保存。 INT4でも高精度。

KV Cache : 自己回帰生成で以前のK、Vを再利用。各ステップの複雑さ $O(L^2) \rightarrow O(L)$ 。

PagedAttention: vLLM のページ単位の KV 管理。マルチシーケンススループット2~4倍向上。

E.4 ソートとファインチューニング

1巻モデルは「次のトークン予測」のみです。チャットボットにするにはアライメントが必要。

SFT (Supervised Fine-Tuning) : コマンドと応答のペアで指導学習。基本チャットボット行動学習。

RLHF: 報酬モデル+ PP0で好みを最適化。 ChatGPTが使用。

DPO (Direct Preference Optimization) : 補償モデルなしで直接の好みを最適化します。シンプルで効果的。
Zephyr、Llama-2-Chatを使用。

LoRA : 低ランクアダプタでファインチューニングコスト99%節約。 7Bモデルを200MBアダプターでファインチ

E.5 データパイプライン

品質フィルタ : 長さ、言語、繰り返し率で低品質のテキストを削除します。

MinHash : Jaccard類似度近似による重複文書の検出。 Web データの 30~50% が重複しています。

合成データ : 強力なモデルで特定のドメインデータを生成します。 Evol-Instruct、Self-Instructなど。

E.6 2巻学習準備

2巻は1巻のコードに基づいて拡張する。 1冊のGPTモデルをインポートします。

- ・ 分散学習で包み (2~第4章)
- ・ アテンションをGQAに置き換えて (5章)
- ・ FFNをSwiGLUに置き換え (5枚)
- ・ LoRAアダプターを追加します (第8章)。

1冊を慣れて扱えば2冊も自然に追いつくことができる。 1冊のコードを直接修正してみて実験することをお勧め

E.7 推奨学習パス

1. 1巻のコードで小さなモデルを学ぶ (scripts/tiny_train.py) 。
2. 自分の言葉で実験 (ウィキバックと韓国語、GitHubコードなど) 。
3. 2巻に渡って分散学習環境を構築。
4. LoRAによるファインチューニング実験 (scripts/lora_finetune.py) 。
5. 量子化による推論の最適化 (scripts/quantize_demo.py) 。
6. 実際のHuggingFaceモデルでサービング練習 (vLLM、TGI) 。

このパスをたどると、ChatGPTレベルのLLMを理解し運営する能力があります。コーディングを停止しないで